AN EXPLAINER

Inside a Gomoku Engine

How a small program looks fifteen plies ahead in half a second, and how it taught itself to judge a position

How to read this document

Every colored tab means the same thing everywhere in this document. Learn the code once and you can skim by color.

THE SIMPLE VERSION	A plain-words picture of the idea, before any detail.
THE CRUX	The one question the chapter answers.
HISTORY	Where the problem came from and the first attempt.
NEW CONSTRAINT	A real-world rule that breaks the previous solution.
WAIT, A QUESTION	The question you were probably about to ask.
THE GOOD	Strengths of an approach.
THE BAD	Weaknesses, costs and pitfalls.
ASIDE / TAKEAWAYS	Side notes, tips and chapter summaries.
blue bold word	A term defined in the glossary at the end.

Contents

1 Introduction 2
2 The big picture 3
3 The board 4
4 Line patterns 9
5 Alpha-beta search 13
6 Making the search deep 18
7 The value network 24
8 Integers all the way down 29
9 The learned policy 33
10 Self-play and Elo 37
11 State of the art review 42
Glossary 44
References 46

1 CHAPTER 1 Introduction

A DIALOGUE

STUDENT A friend of mine wrote a Gomoku program and it beats me every single game. How does it see five-in-a-row coming before I do?

PROFESSOR It doesn't see better than you. It looks at a few million positions before every move, and it has a very fast way to guess who is winning in each of them.

STUDENT A few million? In half a second?

PROFESSOR In half a second. And here's the funny part: almost all of the cleverness is about the positions it decides *not* to look at.

STUDENT And the "guess who is winning" part, somebody wrote that by hand?

PROFESSOR It used to be. Now a small neural network does it, trained on games the engine played against itself. Let's open the thing up.

Gomoku is the game where two players take turns putting stones on a 19×19 grid and the first to line up five wins. This repository plays a harder version of it: you can also **capture** pairs of enemy stones, capturing five pairs wins, and some moves are banned outright. The program answers in under 500 milliseconds, searches 13 to 19 plies deep, and runs on an ordinary laptop.

This document explains the two parts that make it strong: the engine (`src/engine/`), which searches, and the **model** (`train/` plus `nnue.zig` and `policy.zig`), which judges positions and ranks moves. We skip the terminal UI entirely. It works, it's pretty, and there is nothing in it you can't guess.

By the end you should be able to open any file in `src/engine/` or `train/` and say why it exists and what would break without it. You won't need to know Zig. When we point at code, it's so you can go look, not because you have to.

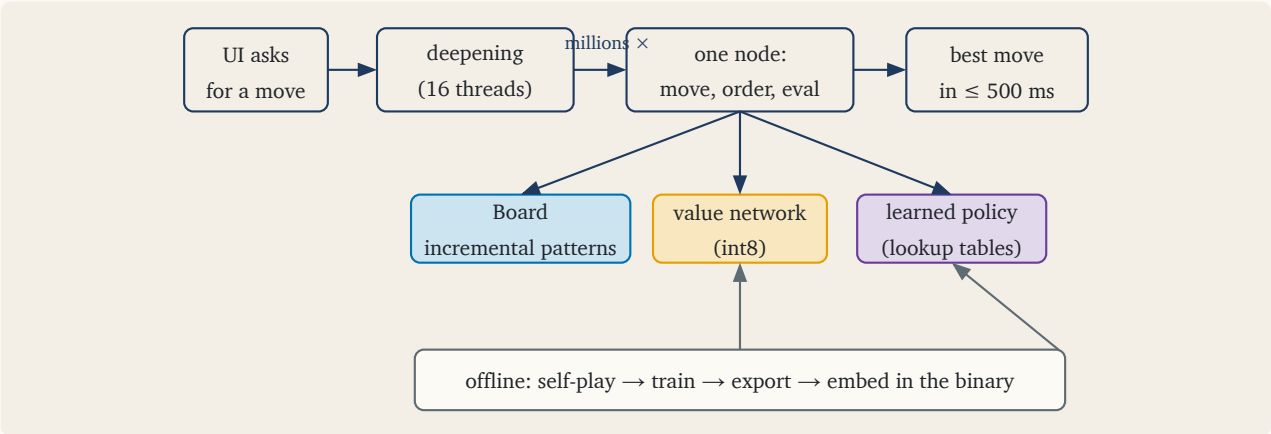


FIGURE 1 *The life of one move, and our roadmap. Everything on the top row happens while you wait; the bottom row happened weeks earlier on a GPU. The three colored boxes are the heart of this document.*

Figure 1 is the whole story in one picture, and the chapters follow it from the bottom up. First we need a board that answers questions fast (chapters 3 and 4). Then a search that looks ahead (chapters 5 and 6). Then the model: a network that judges positions (chapters 7 and 8) and a policy that ranks moves (chapter 9). Last, where the training data comes from and how we know the model helps (chapter 10).

2 CHAPTER 2 The big picture

Before we zoom in, here is every idea in one or two sentences. Don't try to understand them yet. Just notice which ones lean on which.

- **The board.** A 19×19 grid stored with a ring of walls around it, plus a pile of bookkeeping that is updated a little on every move instead of being recomputed from scratch.
- **Line patterns.** For every empty cell and each of the four line directions, the engine knows what the stones around it would become if someone played there: a five, an open four, a three and so on. It's one table lookup.
- **Alpha-beta search.** To pick a move, imagine every move, every reply, every reply to that, and skip any branch you can already prove won't change the answer.
- **Search speed-ups.** Try the likely moves first, look at unlikely ones less deeply, look only at forced moves when someone threatens to win, and run sixteen copies of the search in parallel.
- **The value network.** A tiny neural network that looks at the line patterns and outputs "probability the side to move wins". It's built so one move only changes a few numbers inside it.
- **Integer arithmetic.** The network runs with 8- and 16-bit integers and gives exactly the answer the trainer computed, bit for bit.
- **The learned policy.** A linear model, just a sum of table lookups, that predicts which move the search will end up liking, so the search tries it first.
- **Self-play and Elo.** The engine plays itself hundreds of thousands of times, the games become training data, and matches against the old engine measure whether the new model is really stronger.

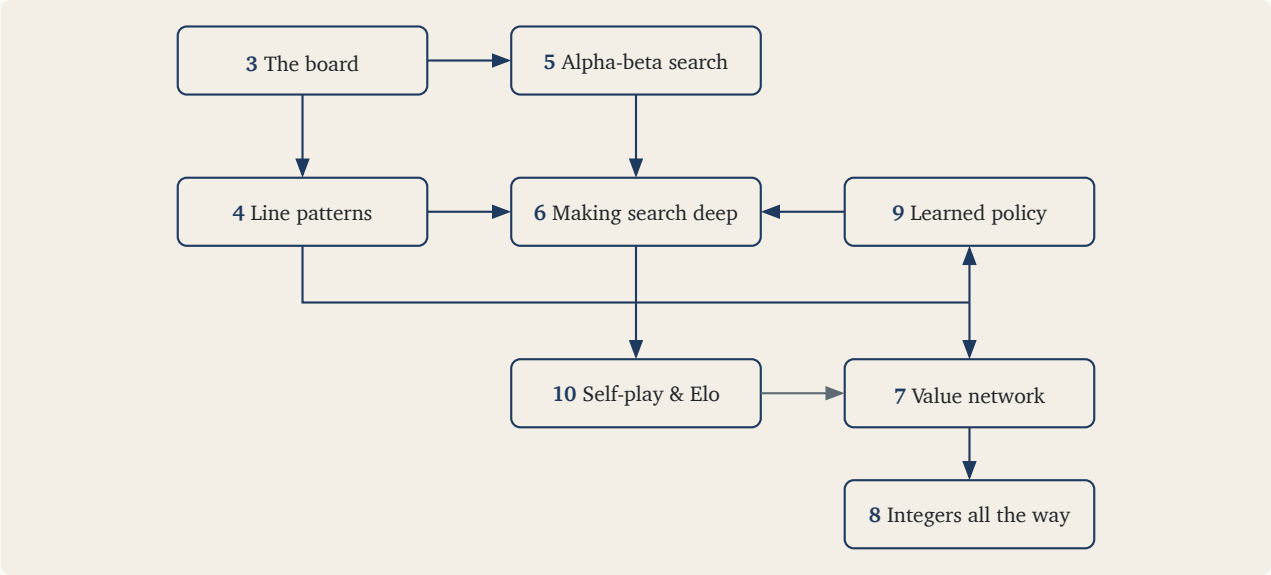


FIGURE 2 *Which chapter needs which. Line patterns (4) feed almost everything: the search's tactics, and through the line along the middle, the inputs of the network and the policy. Self-play (10) closes the loop: the search produces the data the network learns from.*

The surprising thing in Figure 2 is how much hangs off chapter 4. The line patterns are the engine's vocabulary. The search uses them to spot threats, and the network and policy use them as their inputs. Get those right and fast, and everything else gets easier.

3 CHAPTER 3 The board

IN THIS CHAPTER

3.1 The naive solution
- *the edges are everywhere*
- *the search asks the same question a million times*
- *every move must be taken back*

3.2 How it really works

3.3 State of the art

"In go-moku, simple rules lead to a highly complex game, played on the 225 intersections of 15 horizontal and 15 vertical lines."
– Allis (1994)

? **THE CRUX**

How do we store a Gomoku position so that the questions the search asks millions of times per second ("is this a four?", "can I capture here?") cost almost nothing?

THE SIMPLE VERSION

Imagine a stadium scoreboard. You could recount every fan in every seat each time someone asks "how many people are here?", or you could add one when someone walks in and subtract one when someone leaves. The second way is the board in this engine: it never recounts, it only adjusts.

HISTORY: THE ORIGINAL PROBLEM

Five-in-a-row games are centuries old, and the plain version is too easy for the first player: in his 1994 thesis, Victor Allis proved by computer search that, on a 15×15 board, the first player can always force a win (Allis, 1994; Allis, Herik and Huntjens, 1996). Variants fight this in two ways. Renju bans certain shapes for the first player, including the double three (Renju International Federation, 1998). Pente and Ninuki-renju add captures: flank two enemy stones and you remove them (Wikipedia contributors, 2026). The rules this repository implements, from a 42 school assignment, borrow both ideas.

Let's pin the rules down, because every one of them shows up in the code:

- Five or more stones in a row win.
- Flanking exactly two enemy stones (`X O O X`) captures them. Capturing five pairs wins.
- A five that the opponent can break by capturing one of its stones doesn't win yet: the opponent gets one move to break it.
- A move can't create two free threes at once (a "double three"), unless it captures or completes a five.

3.1 The naive solution

The obvious representation is a 19×19 array of bytes: 0 for empty, 1 for black, 2 for white. To check for a win after a move, walk from the new stone in the four line directions and count. To score a position, scan the whole board and look for shapes.

This works, and for a program that thinks for a second about one position, it's fine. We're pretending two things don't matter: that the search will ask these questions **millions** of times per move, and that the edges of the board exist.

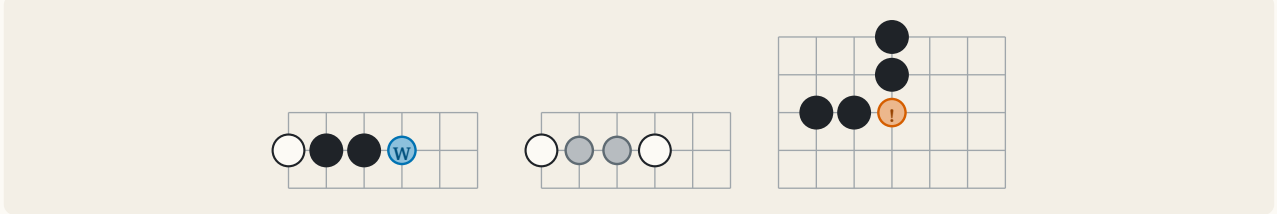


FIGURE 3 *Left: white playing at W flanks the black pair. Middle: the pair is gone and white has one captured pair. Right: black playing at ! would make two open threes at once, a double three, so the move is illegal unless it also captures.*

NEW CONSTRAINT: THE EDGES ARE EVERYWHERE

Every walk along a line has to ask "am I still on the board?" at each step. That's a branch in the hottest loop of the program, and near the edges the patterns need special cases.

The fix is an old chess-programming trick called a **mailbox** board. Put a ring of wall cells, four deep, around the 19×19 grid, giving a 27×27 array of 729 cells. A walk along a line can now go four steps in any direction from any real cell and it lands on a wall, never outside the array. Four is not arbitrary: it's exactly as far as a five-in-a-row pattern can reach from its centre. Walls are just another cell value, so the pattern code treats "the edge" and "an enemy stone" with the same machinery.

A move is then a single number, the index into that padded array (`Board.Move` is a `u16`). Index 0 is a wall, so 0 doubles as "no move".

NEW CONSTRAINT: THE SEARCH ASKS THE SAME QUESTION A MILLION TIMES

Searching 15 moves deep means millions of positions, and each one needs to know where the fours and threes are. Rescanning 361 cells in four directions each time would eat the whole time budget before the search got anywhere.

Here's the key observation. A stone only affects the patterns of cells within four steps of it along its four lines. That's 4 directions × 8 neighbours = **32 cells**. Every other cell's situation is exactly what it was before. So instead of recomputing, the board keeps everything up to date by adjusting only those 32 cells.

To make even those 32 updates cheap, the board stores each line of the board (every row, column and both diagonals) as a 64-bit integer, two bits per cell. Pulling out the nine-cell window around any cell is then a shift and a mask, not a loop. The eight windows along one line all come from the same 64-bit word, so `reclassifyNeighbours` in `Board.zig` computes them with one vector shift. We'll see in the next chapter what a window turns into.

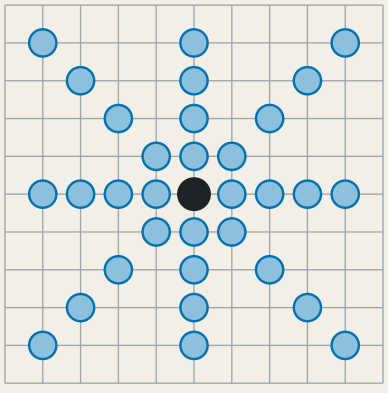


FIGURE 4 *The 32 cells (blue) whose line patterns can change when a stone is placed in the middle. The other 328 cells of a 19×19 board are untouched, which is why updating beats recomputing.*

NEW CONSTRAINT: EVERY MOVE MUST BE TAKEN BACK

The search plays a move, looks at the result, and takes it back, millions of times. Taking a stone off the board means re-deriving the patterns of 32 cells, and if the move captured, of every captured stone's neighbours too.

The fix is an **undo log**. Before `makeMove` overwrites anything (a cell's record, a line key, a bitboard word, the running sums, the network accumulator) it copies the old value into a per-thread log. `undoMove` just copies them back, newest first. Copying 16-byte records back is much cheaper than reclassifying patterns, which is exactly the comment in the code: "much cheaper than recomputing".

WAIT, A QUESTION

Q *If everything is updated incrementally, how do you know it hasn't drifted out of sync after a million moves?*

A You don't, unless you check. `Board.matchesRecomputedState` rebuilds every structure from the raw stones and compares it field by field with the incremental version. The tests play long random games and call it after every move. Incremental code without this kind of check is a bug farm.

3.2 How it really works

Each of the 729 cells has a 16-byte `CellRecord`. It holds, for both colors at once:

- the line information for each of the four directions (what pattern that color would make there, and whether playing there captures, threatens a capture or leaves a pair capturable),
- the combined verdict over all four directions (one of 13 categories, the next chapter's `Pattern4`),
- the number of captures available there, and a hand-tuned score used by the classical evaluation.

Sixteen bytes means four cells per 64-byte cache line, so the update loop touches as little memory as possible. The comment in the code explains why the fields are grouped by cell rather than in separate arrays: the update loop reads and writes all of them for both colors at once.

THREAT BITBOARDS

Some questions are about the whole board: "does the opponent have any five-point?" or "do I have an open four anywhere?". Scanning 361 records for that would be slow again. So the board also keeps **bitboard**s: for each color, one bit plane per strong category (double three and up) plus one plane for capture points. 729 cells fit in 12 words of 64 bits. "Does white have a five-point?" becomes an OR over 12 words. When a cell's category changes, `updateThreatBits` moves its bit from one plane to another.

There's a third bitboard: **candidate moves**. A move far from every stone is almost never good, so the engine only considers empty cells in the 5×5 square around a stone, plus cells exactly three steps away along a line, where long-range threats start. That's 32 offsets per stone, updated as stones come and go. In a typical middle game this cuts 300-odd empty cells down to a few dozen.

HASHING THE POSITION

Finally, the board keeps a 64-bit fingerprint of the position, the **Zobrist hash** (Zobrist, 1970). Each (color, cell) pair gets a random 64-bit number, and the hash is the XOR of the numbers of every stone on the board, plus one per captured-pair count. Placing or removing a stone is one XOR. We'll need this fingerprint in chapter 5 to recognise positions we've already searched.

3.3 State of the art

Padded boards, incremental pattern tables and Zobrist hashing are the standard toolkit of strong Gomoku programs. Rapfi, which won the 2024 Gomocup computer tournament (Jin, Duan and Hang, 2025) and most leagues of the 2026 one (Gomocup, 2026), also reads the board as line patterns around each point, four directions per point, and updates what it derives from them incrementally when a stone is placed (Jin, Duan and Hang, 2025). Chess engines use the same ideas with different shapes: bitboards fit chess well because 64 squares fit one word, while the padded 729-cell Gomoku board needs 12 words per plane, which is why this engine uses bitboards only for the few sparse sets it scans whole-board.

The thing that changes from engine to engine is **how much** to keep incrementally. Keep too little and you recompute; keep too much and every move pays to update things nobody reads. This engine keeps exactly what the search and the evaluation read on every node, and nothing else.

THE GOOD

+ A move updates 32 neighbour cells instead of rescanning 361
+ No bounds checks: walls are just a fourth cell value
+ Undo is a memory copy, not a recomputation
+ Whole-board threat questions are an OR over 12 words

THE BAD

– About 1,500 lines of tricky bookkeeping in `Board.zig`
– Every incremental structure can drift, so it needs a from-scratch checker and tests
– Captures remove up to 16 stones in one move, each with its own 32 neighbours

KEY TAKEAWAYS

1. The board is a 27×27 array: the walls remove every bounds check.
2. A stone changes only the 32 cells on its lines within four steps, so everything is updated, never recomputed.
3. An undo log makes taking a move back a copy.
4. Bitboards answer whole-board threat questions; a Zobrist hash fingerprints the position.

4 CHAPTER 4 Line patterns

IN THIS CHAPTER

4.1 The naive solution
- *the shapes are subtler than counting*
- *we can't afford the recursion at runtime*

4.2 How it really works

4.3 State of the art

"Since line connections are crucial in Gomoku, we organize these local patterns into line segments in various directions ..."
– Jin, Duan and Hang (2025)

THE CRUX

How do we turn "the stones around this empty cell" into a verdict like "playing here makes an open four", using one memory read?

THE SIMPLE VERSION

Think of a phrasebook for tourists. You don't work out the grammar of a sentence each time; you look the whole sentence up. Every possible short stretch of a line on the board is a "sentence", and the engine has a book with the meaning of every one of them.

HISTORY: THE ORIGINAL PROBLEM

Gomoku strength is all about threats. When Allis's team solved free-style Gomoku, their key tool was **threat-space search**: only look at moves that create fours and threes, because the defender has to answer them (Allis, Herik and Huntjens, 1996). To do that, a program must first recognise threats instantly and correctly, including the awkward broken shapes.

4.1 The naive solution

The direct approach: for each empty cell and each direction, walk outwards, count consecutive stones, check whether the ends are open, and run a chain of `if`s. "Four in a row with both ends open: open four. Three with both ends open: open three."

We're ignoring two things here: that the real shapes are subtler than counting, and that this is far too slow to run on every cell of every node.

NEW CONSTRAINT: THE SHAPES ARE SUBTLER THAN COUNTING

`X X . X X` has no four consecutive stones, yet filling the hole makes five: it's a four. `. X X X . #` with a wall two steps on can look open and still be dead. Hand-written rules for every case are long, and they are wrong in ways you only find by losing.

The fix is to define patterns by what they can **become**, recursively. This is the definition `pattern.zig` uses, which the comment credits to Rapfi. Look at a nine-cell window: the empty cell in the middle and four cells on each side. Then, from black's point of view, with black about to play in the middle:

- **F5** (five): playing the centre makes five or more in a row.
- **F4** (open four): playing the centre creates **two** different cells where one more stone makes five. The opponent can block only one.
- **B4** (blocked four): it creates exactly one such **five-point**.
- **F3** (open three): playing the centre makes a shape where one more move gives an F4.
- **B3**: one more move gives a B4.
- and so on down: **F2** if one more move gives an F3, **B2** for a B3, and **F1** and **B1** for the shapes one step further away, until **dead**: this line can never make five.

Look at row 4 of Figure 5. A counting rule would call it a mess. The recursive rule just tries every empty cell, sees two of them complete five, and says F4. No special case needed. The definition is short and you can check it exhaustively, which is the real win.

NEW CONSTRAINT: WE CAN'T AFFORD THE RECURSION AT RUNTIME

Classifying one window means trying every empty cell and classifying the result, recursively. Doing that during the search would be absurdly slow.

The fix: precompute **every possible window**. Each of the 8 neighbours is one of four codes (empty, black, white, wall), 2 bits each, so a window is a 16-bit number and there are $4^8 = 65{,}536$ of them. Only $121 \times 121 = 14{,}641$ can occur on a real board, since walls only show up as a solid block at the ends, but a 64K table indexed directly is simpler. A build-time tool, `src/tools/gen_tables.zig`,

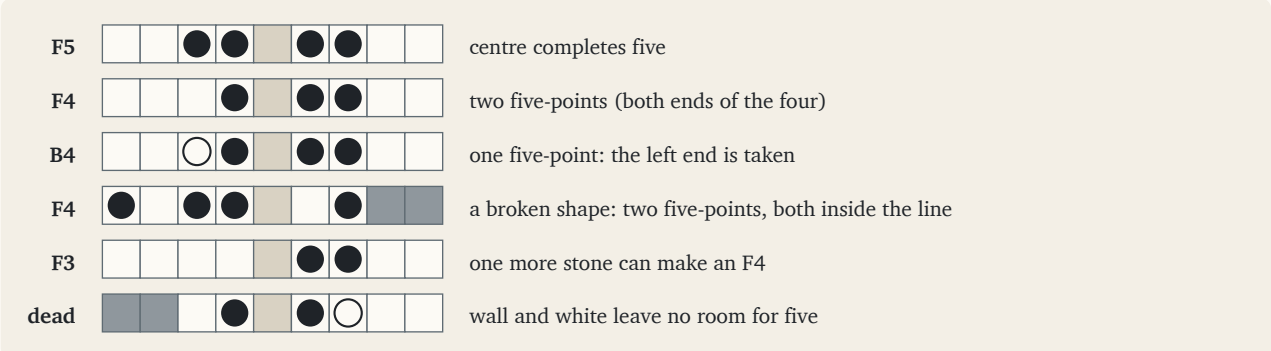


FIGURE 5 *Pattern classes for black playing the shaded centre cell. Each class is defined by what one more move can turn it into, which is why broken shapes like row 4 come out right without any special case. Gray cells are the wall past the board's edge.*

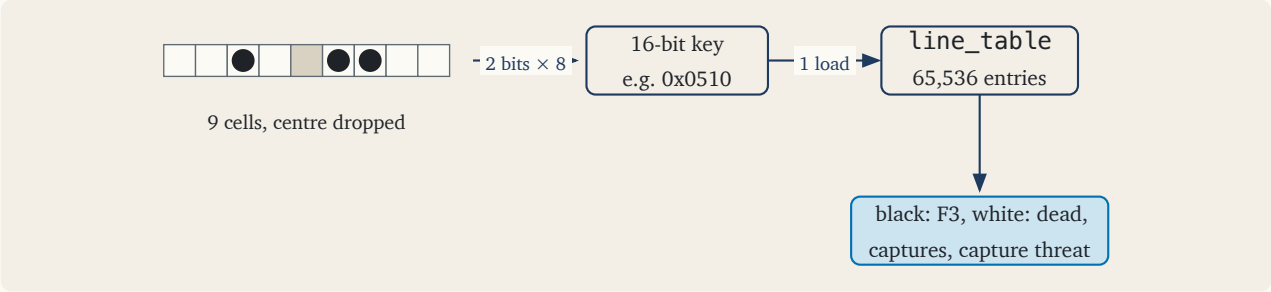


FIGURE 6 *From stones to a verdict. The window around an empty cell becomes a 16-bit number, and that number indexes a precomputed table holding the pattern for both colors plus capture information. No loops, no branches.*

classifies all of them and the result is embedded in the binary. At runtime, classification is one load from a 128 KB table.

The table entry carries a bit more than the pattern. Each color gets one byte, a `LineInfo`: 4 bits of pattern, 2 bits for how many pairs playing here captures on this line, one bit "playing here threatens a capture", and one bit "playing here leaves my own pair capturable". Captures in this game are line shapes too (`X O O X`), so they fit in the same lookup.

WAIT, A QUESTION

Q *Why drop the centre cell from the window? Isn't it important?*

A We only classify **empty** cells, so the centre is always empty and carries no information. Dropping it saves two bits, which makes the table 4× smaller and keeps it in the L2 cache. The code even notes that a cleverer, more compact table was tried and came out 3% slower on an Apple M5: one table that fits in cache beats two dependent lookups.

4.2 How it really works

One direction isn't the whole story. A cell can be an F3 horizontally and a B4 diagonally, and the combination matters more than either. So the engine combines the four directional patterns into one of 13 **Pattern4** categories, with a second precomputed table. The four patterns are packed as four 4-bit nibbles into a 16-bit index, so combining is again one load.

Table 1 also tells us how the double-three ban is checked: a cell is a double three for a color when two of its directions are F3. `Board.isForbidden` adds the exceptions (the move completes a five, or it captures) and that's the whole rule, at the cost of reading two bytes that are already in the cell record.

THE CLASSICAL EVALUATION

These patterns were enough to build a decent player before any neural network existed, and the repository keeps that player as a baseline. Its evaluation (`eval.zig`) gives each directional pattern a hand-picked value, from 1 for a B1 up to 400 for an F5, adds bonuses for Pattern4 categories, and adds terms for threats and captures. Some of its weights, straight from the source: an open four for the side to move is worth 2,500, a four-three 1,200, five captured pairs 3,000.

Where do those numbers come from? A human guessed them and tweaked them until the engine played better. Keep that question in mind; chapter 7 is the answer to it.

Category	Made of (over the four directions)	Means
`a_five`	any F5	wins now
`b_flex4`	an F4, or two B4s	wins next move: two five-points
`c_block4_flex3`	a B4 and an F3	the classic “four-three” win
`d_block4_plus`	a B4 and a B3 or F2	forcing, with follow-up
`e_block4`	a lone B4	forcing: one reply
`f_flex3_2x`	two F3s	double three: banned, or a win
`g_flex3_plus` … `l_flex2`	weaker mixes of threes and twos	building material
`none`	nothing useful	

TABLE 1 *The 13 Pattern4 categories, strongest first (`combine` in `pattern.zig`). Notice that “two blocked fours” is ranked with the open four: two five-points are two five-points, whichever lines they’re on.*

4.3 State of the art

Precomputed line tables are how the strongest Gomoku engines start. Rapfi goes one step further than hand-made categories: its network looks at a length-11 line segment in each of the four directions around a point, and looks each segment up in a precomputed table whose values were distilled from a larger convolutional network (Jin, Duan and Hang, 2025). This engine’s value network (chapter 7) does something close: besides the 13 hand-made categories, it reads the raw 16-bit window of each direction and learns a table row for every window, so it can learn distinctions the categories throw away.

THE GOOD

+ Classifying a cell is one load from a 128 KB table
+ The recursive definition handles broken shapes and walls with no special cases
+ Captures, capture threats and the double-three ban come out of the same lookup

THE BAD

– The window sees four cells each way: threats that span further are invisible to it
– Categories are fixed by hand; the 13 buckets merge positions a strong player would tell apart
– Tables must be regenerated, and the network retrained, if the definition changes

KEY TAKEAWAYS

1. A pattern is defined by what one more stone turns it into: five, two five-points, one five-point, and so on down.
2. Every possible nine-cell window is classified at build time; at runtime it's one lookup.
3. Four directional patterns combine into one of 13 Pattern4 categories, again by lookup.
4. These patterns are the vocabulary of the search, the network and the policy.

5 CHAPTER 5 Alpha-beta search

IN THIS CHAPTER

5.1 The naive solution
- *the tree is astronomically big*
- *we don't know how deep we can afford to go*
- *the same position shows up again and again*

5.2 How it really works

5.3 State of the art

"A strategy for chess may be described as a process for choosing a move in any given position."
– Shannon (1950)

THE CRUX

How do we choose a move when its value depends on what the opponent replies, what we reply to that, and so on, without examining every possible future?

THE SIMPLE VERSION

You're planning a move and you imagine "if I go here, they go there, then I go…". The moment you find one reply that ruins a plan, you stop thinking about that plan: it's already worse than one you've found. You don't need to know **how** bad it is.

HISTORY: THE ORIGINAL PROBLEM

In 1950, Claude Shannon described how a computer could play chess: look a few moves ahead, score the positions at the end with an evaluation function, and assume both sides pick their best option (Shannon, 1950). Researchers quickly noticed that whole branches could be skipped without changing the result, and in 1975 Donald Knuth and Ronald Moore gave the

first full analysis of this "alpha-beta" pruning, including how much it saves when moves are tried in the best order (Knuth and Moore, 1975).

5.1 The naive solution

Picture the future as a tree. The root is the current position, its children are the positions after each of our moves, their children the positions after each reply, and so on. At the leaves, at some fixed depth, we call the evaluation and get a number: positive is good for us.

Minimax fills the tree in from the bottom. At our turn we take the maximum of the children; at the opponent's turn, the minimum. The value at the root is what we can guarantee against perfect play, at that depth.

We're pretending the tree is small. It isn't.

NEW CONSTRAINT: THE TREE IS ASTRONOMICALLY BIG

Even with the candidate filter from chapter 3, a middle-game position has around 40 sensible moves. Looking 10 **ply** deep means $40^{10} \approx 10^{16}$ leaves. At 7 million positions per second, that's about 48 years for one move.

ALPHA-BETA PRUNING

Here's the trick from the simple box, made precise. While searching, carry two numbers. **Alpha** is the score we're already guaranteed somewhere else in the tree; beta is the score the opponent is already guaranteed. If a position's score is proven to be at least beta, the opponent will never let us reach it, so we can stop searching it. That's a **cutoff**.

How much does it save? Knuth and Moore showed that if the best move is always tried first, alpha-beta looks at about $b^{d/2}$ leaves instead of b^d (Knuth and Moore, 1975). In other words, it searches **twice as deep** in the same time. With bad ordering it saves almost nothing. So the whole game becomes: try the best move first. Chapter 6 is mostly about that.

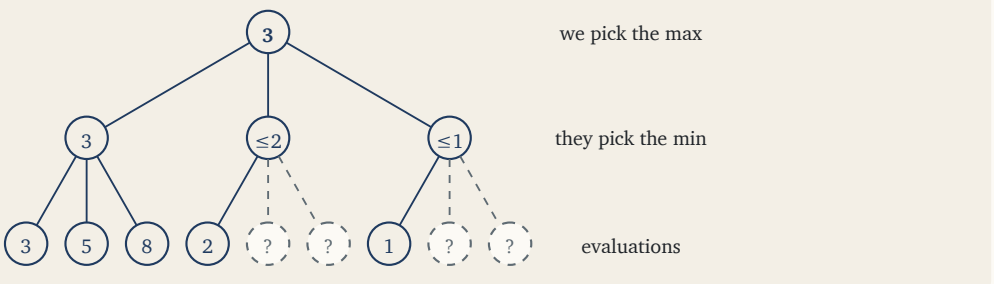


FIGURE 7 *Alpha-beta on a tiny tree. After the left branch we're guaranteed 3. In the middle branch the opponent's first reply already holds us to 2, so its other replies (dashed) can't matter: we'd never choose this branch anyway. Four of the nine leaves are never evaluated.*

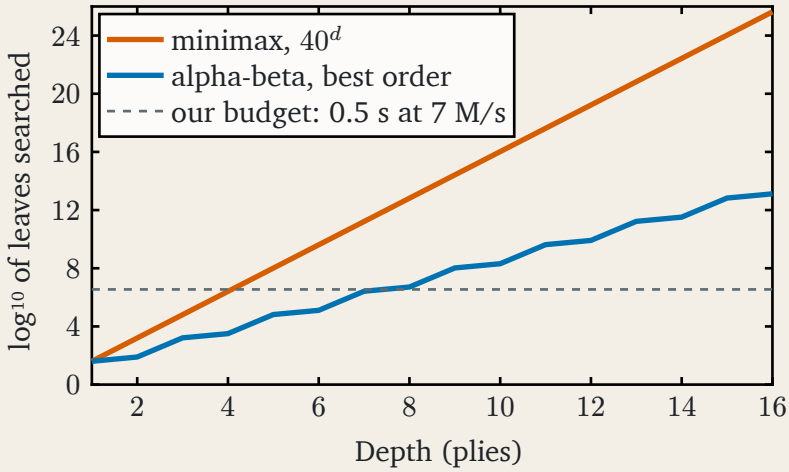


FIGURE 8 *Leaves searched as depth grows, for a uniform branching factor of 40 (an illustrative round number). Minimax hits the half-second budget around depth 4; perfectly ordered alpha-beta around depth 8. The engine really reaches 13 or more, so there must be more tricks: that's chapter 6.*

NEW CONSTRAINT: WE DON'T KNOW HOW DEEP WE CAN AFFORD TO GO

The time budget is fixed (under 500 ms per move) but the size of the tree isn't: a quiet opening and a sharp tactical fight differ by orders of magnitude. Pick depth 12 and you may run for a minute; pick 8 and you waste most of the time.

The fix is **iterative deepening** (Slate and Atkin, 1977). Search to depth 1, then 2, then 3, and keep going until time runs out. That sounds wasteful, but each depth costs many times more than all the previous ones put together, so the repeats are cheap. And it pays for itself: the best move found at depth d is an excellent first guess at depth $d+1$, which is exactly the ordering that makes alpha-beta fast.

The engine stops in two ways (`isDoneAfterIteration` in `search.zig`). There's a soft limit, past which no new depth is started, and a hard limit, which interrupts the search in the middle. There's also a nice heuristic: if the same best move came out of six iterations in a row and more than half the time is used, it won't change, so stop.

NEW CONSTRAINT: THE SAME POSITION SHOWS UP AGAIN AND AGAIN

Playing A then B reaches the same position as B then A. In Gomoku, where stones don't move, this happens all the time: a position at depth 6 can be reached by dozens of move orders. Plain alpha-beta searches each one from scratch.

The fix is a **transposition table** (TT): a big hash table keyed by the Zobrist hash from chapter 3. After searching a position, store its score, the depth it was searched to, whether the score is exact or only a bound, and the best move. Before searching a position, look it up. If it was searched deep enough, reuse the score. If not, at least try its stored best move first. That last part is worth as much as the first.

WAIT, A QUESTION

Q *If two different positions get the same 64-bit hash, won't the engine reuse the wrong score?*

A It can happen, but with 64 bits it's rare enough to ignore: even if the table sees billions of positions per game, the chance a given lookup hits a different position with the same full key is tiny. Engines accept that risk; the alternative, storing the whole board in each entry, would make the table many times smaller.

5.2 How it really works

The engine's search (`Worker.search` in `search.zig`) is alpha-beta written in **negamax** form: the score of a position for the side to move is the negation of its children's scores. One function, no separate min and max.

PRINCIPAL VARIATION SEARCH

On top of plain alpha-beta, it uses **principal variation search** (PVS), introduced by Marsland and Campbell in 1982 and essentially equivalent to Reinefeld's NegaScout (Chessprogramming wiki, no date a; Reinefeld, 1983). The bet is that the first move, ordered by the TT and the policy, is the best one. So the first move gets a normal search with the window (alpha, beta). Every other move only gets a **null window** (alpha, alpha + 1), which can't return an exact score, just "not better than alpha" or "better". A null-window search cuts off much more often, so it's cheap. If one does say "better", we re-search that move with the full window.

WAIT, A QUESTION

Q *Why search with a window that can only say "better or not"? Don't we want the actual score?*

A For moves that turn out worse than the first, no: you only need to know they're worse, and proving "worse" is much cheaper than measuring by how much. Most moves are worse, so most searches are cheap. The occasional re-search is the price.

ASPIRATION WINDOWS

From depth 5 on, iterative deepening doesn't start with an infinite window either. It assumes the score will be close to the previous iteration's and searches with a window of ±25 around it. If the true score falls outside, the window doubles on that side and we search again; past ±1,500 it gives up and opens fully (`aspirationSearch`). Narrow windows mean more cutoffs everywhere in the tree.

SCORES FOR WINS

A win isn't just a big number. The engine scores "I win in n plies" as $30{,}000 - n$, so it prefers the fastest win and the slowest loss. Anything above 29,000 is a proven result, which static evaluations (clamped to ±20,000) can never reach. The TT has a subtlety here: the same position can sit at different distances from the root, so mate scores are stored relative to the position, not the root, and converted back when read (`toStoredValue` and `fromStoredValue`).

5.3 State of the art

This skeleton (negamax alpha-beta, PVS, iterative deepening with aspiration windows, a transposition table) is what top chess engines have used for decades (Chessprogramming wiki, no date a), and it's what this engine uses. The other big family is Monte Carlo tree search guided by a neural network, made famous by AlphaZero (Silver *et al.*, 2018) and used by KataGo (Wu, 2019). MCTS grows the tree selectively and averages results instead of proving bounds.

So which one fits Gomoku on a laptop? It shines when the evaluation is a large, slow, accurate network run on a GPU. Alpha-beta shines when the evaluation is tiny and fast and the game is tactical, where one missed reply loses on the spot. Gomoku with captures is very tactical, and this engine runs on a CPU in 500 ms, which is why alpha-beta is the natural choice. Rapfi, the strongest engine at recent Gomocups, is also an alpha-beta engine (Jin, Duan and Hang, 2025; Gomocup, 2026).

THE GOOD

+ With good ordering, about twice the depth of minimax in the same time
+ Exact: pruning never changes the minimax result
+ Iterative deepening gives a usable answer whenever time runs out
+ The TT turns repeated positions into lookups and supplies the best first move

THE BAD

– With bad ordering, alpha-beta saves almost nothing
– Still exponential: each extra ply multiplies the cost
– Everything beyond the horizon is invisible unless something extends it

KEY TAKEAWAYS

1. You don't need to know how bad a refuted plan is, only that it's refuted: that's all of alpha-beta.
2. Alpha-beta's savings depend on trying the best move first: the theme of the next two chapters.
3. Iterative deepening fits any time budget and feeds its own move ordering.
4. In a game where stones never move, the same position arrives by many paths; hashing it turns repeats into lookups.

6 CHAPTER 6 Making the search deep

IN THIS CHAPTER

6.1 The naive solution
- *most moves are obviously bad*

6.2 Spending less on bad moves
- *Gomoku is a game of forced moves*
- *the horizon cuts tactics in half*
- *one core is not enough*

6.3 How it really works

6.4 State of the art

"LMR can often reduce the effective branching factor to less than 2, depending on the reduction conditions."
– Chessprogramming wiki, "Late Move Reductions" (Chessprogramming wiki, no date b)

THE CRUX

How do we reach 13 to 19 plies in half a second, when even perfectly ordered alpha-beta would stall around 8?

THE SIMPLE VERSION

A strong player doesn't think about every move equally. They look hard at the two or three moves that matter, glance at the rest, and ignore anything silly. And when someone threatens to win, they stop thinking about anything except how to stop it.

HISTORY: THE ORIGINAL PROBLEM

Chess programmers spent the 1970s to the 2000s finding ways to cheat the exponential. Statistics such as Schaeffer's history heuristic order moves using what worked elsewhere in the tree (Schaeffer, 1989). Null-move pruning, popularised by Donninger in 1993, asks "if I pass and am still winning, why look further?" (Donninger, 1993). Late move reductions, which look at late moves less deeply, became standard in the 2000s (Chessprogramming wiki, no date b). Gomoku adds its own gift: it's a game of forced moves (Allis, Herik and Huntjens, 1996).

6.1 The naive solution

Take chapter 5's search and add good ordering: TT move first, then everything else in a sensible order. With perfect ordering alpha-beta reaches about depth 8 in our budget, as Figure 8 showed. We're ignoring that real positions aren't uniform: most moves are bad, some positions are forced, and the interesting stuff often happens just past the horizon.

NEW CONSTRAINT: MOST MOVES ARE OBVIOUSLY BAD

Alpha-beta treats every move that isn't cut off with the same respect. The 25th move in a well-ordered list almost never turns out best, yet it gets the same full-depth search as the 2nd. And in many positions one side is so clearly ahead that searching every move just confirms it.

6.2 Spending less on bad moves

ORDERING

First, order as well as possible. Each candidate gets a score (`movepick.zig` and `search.zig`):

- the TT move goes first, always;
- the two **killer move**s from this depth get a bonus (+6,000 and +5,000);
- a per-thread **history** table adds credit to moves that caused cutoffs anywhere recently (Schaeffer, 1989);
- the **learned policy** (chapter 9) scores what the move does on the board.

Picking is lazy: one step of selection sort per move tried (`selectBest`), because most nodes cut off after one or two moves and sorting the whole list would be wasted. The scores sit in their own dense array, so finding the best is a SIMD max over eight at a time.

LATE MOVE REDUCTIONS AND PRUNING

With ordering in place, the engine bets on it. Moves late in the list get searched to a **smaller** depth. This is **late move reductions** (LMR) (Chessprogramming wiki, no date b). The reduction grows with both depth and move number:

$$R(d, m) = \left\lfloor 0.4 + \frac{\ln d \cdot \ln m}{1.9} \right\rfloor$$

then adjusted: one less in PV nodes or for semi-threats, one more in expected cut nodes and when the position isn't improving, less for moves with good history, and adjusted by the policy (chapter 9). Threats (fours, captures, double threes, blocks of open fours) are never reduced. If a reduced move surprises us by beating alpha, it's searched again at full depth, so reductions cost accuracy only when they're wrong **and** the move looked fine at low depth.

WAIT, A QUESTION

Q *Doesn't pruning and reducing so much make the engine miss things?*

A Yes, sometimes. Every one of these tricks is a bet that's usually right, and the engine re-searches when a bet looks wrong. The test is never "is it exact?" but "does it win more games at the same time limit?". Each trick in this chapter stayed because it passed that test. The policy-based pruning in chapter 9 is an example of one that didn't.

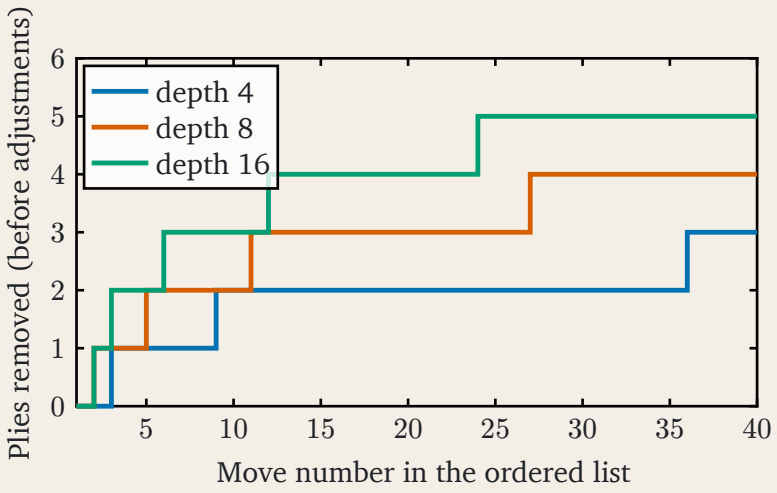


FIGURE 9 *The base LMR table from* `search.zig`*, computed exactly. At depth 16 the 20th move is searched 4 plies shallower. Deeper searches reduce more, which is how the effective branching factor drops far below the number of legal moves.*

Below depth 8 the engine goes further with late move pruning: after a fixed number of quiet moves (5 at depth 1, 8 at depth 2, … 38 at depth 7), the rest are skipped entirely. And at depth ≤ 6, if the static evaluation minus a margin already beats beta, the node returns immediately (reverse futility pruning).

PASSING AS A TEST

What about positions where one side is clearly ahead? The **null move** answers that cheaply (Chessprogramming wiki, no date c; Donninger, 1993). If the side to move is already above beta, let it pass, a move that isn't legal, and search the opponent's reply with a reduced depth ($3 + d/4$ plies less). If even after passing we're still above beta, the position is almost surely good enough, and the node is cut. The engine doesn't do this when it has a four available or the opponent has an open four, because those are exactly the positions where passing is a disaster and the test lies.

> **NEW CONSTRAINT: GOMOKU IS A GAME OF FORCED MOVES**
>
> If the opponent has a five-point, every move except blocking it (or capturing, or winning first) loses immediately. The search above would still generate 40 moves, search the first, and only cut off the rest one by one.

Here Gomoku helps us. The board's threat bitboards tell us instantly whether someone has a five-point or an open four, and the move generator narrows the list:

- **defend five** (`defend_five`): the opponent threatens to win next move. Only our own fives, blocks of the five-point, and captures that break the threat survive, and `keepRealFiveDefences` checks each capture really helps.
- **defend four**: the opponent has an open four, which will become two five-points. Only moves on its lines, our own fours and captures survive, and a quick geometric test drops moves that can't touch any of the open-four points.

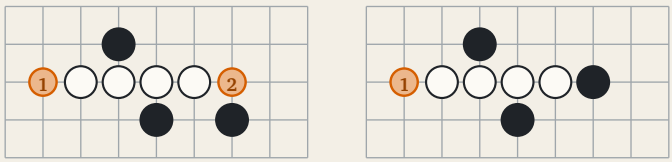


FIGURE 10 *Left: white's open four has two five-points (marked). Black can block one, so unless black has a capture or a faster win, the position is lost, and the search only needs to look at those few moves. Right: once one end is blocked, the four has a single five-point (marked). Black has exactly one reply, so the search looks at one move instead of forty.*

When the defender has exactly one legal reply, the engine also **extends** the search by a ply. A single forced reply costs no branching, and it stops long forcing sequences from falling off the horizon.

> **NEW CONSTRAINT: THE HORIZON CUTS TACTICS IN HALF**
>
> At depth 0 the search calls the evaluation. If that position is in the middle of a sequence of fours, where one side forces the other move by move until it wins, a static evaluation can't see it. The engine would happily walk into a loss two plies past its horizon.

The fix is a **quiescence search** specialised for Gomoku: a **VCF** search, for "victory by continuous fours". At the horizon, the side to move may keep playing fours. Each four has exactly one answer, so the tree stays thin, and tactics get resolved instead of cut off. The engine bounds it at 4 levels, and each level keeps only a small budget of the strongest fours (`quiescence` in `search.zig`).

> **NEW CONSTRAINT: ONE CORE IS NOT ENOUGH**
>
> The laptop this engine was benchmarked on has 18 cores. Alpha-beta is sequential at heart: the result of the first move sets the window for the others. Splitting the tree carefully between threads is famously hard to get right.

The engine uses **Lazy SMP** (Chessprogramming wiki, no date d), which sounds too simple to work: start up to 16 threads, have them all search the same position, and let them share only the transposition table. They help each other through the table: one thread's results become another's cutoffs and move ordering. To keep them from doing identical work, helper threads skip some depths in staggered patterns, so they're often a depth ahead of the main thread.

A table that 16 threads write at once would normally need locks, which would be slow. Instead the engine uses the Hyatt–Mann lockless trick (Hyatt and Mann, 2002): each slot stores `key XOR entry` next to `entry`. If two threads write a slot at the same time and a reader sees half of each, the XOR check fails and the slot reads as a miss. No lock, and a torn write costs one missed lookup.

6.3 How it really works

So what actually happens when the search lands on a node? Here's one call of `search`, in order, with all the pieces in place:

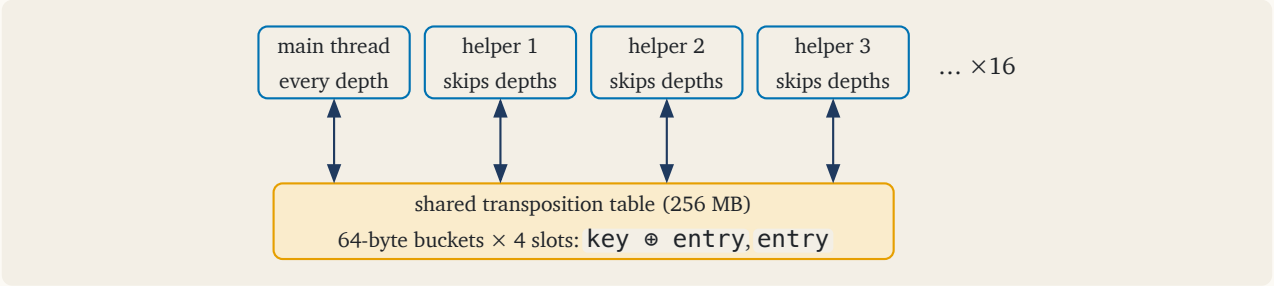


FIGURE 11 *Lazy SMP. The threads never talk to each other, only to the table. A probe touches exactly one 64-byte cache line (four slots), and a torn concurrent write simply fails the XOR check.*

1. **Forced situations first.** If the opponent just made a five that can still be broken by a capture, the only moves are the breaking captures. If the side to move can win by capture or by an unbreakable five, return a win.
2. **Mate distance pruning.** If even the fastest possible win can't beat alpha, stop.
3. **Leaf.** At depth 0, go to the VCF quiescence search.
4. **TT probe.** If the stored result is deep enough and its bound fits the window, return it. Otherwise remember its best move.
5. **Cheap cutoffs**, in quiet non-PV nodes only: reverse futility pruning, then the null move.
6. **Internal iterative reduction.** No TT move at depth ≥ 4 means this node probably wasn't searched before and ordering will be poor, so search it one ply shallower.
7. **Generate moves** in the mode the threats dictate (all, defend four, defend five), then score them: TT move, killers, history, policy.
8. **Loop over moves**: late move pruning, then make the move, check for an immediate win, search with LMR and PVS, undo.
9. **Store** the result in the TT, and reward the move that cut off in the killer and history tables.

In the repository's benchmark (`make bench`), the engine plays itself with a 300 ms soft limit and a 450 ms hard limit. On an Apple M5 Max with 16 threads, it averaged about 7 million positions per second, no move took longer than 450 ms, and every search that hadn't already found a forced result reached depth 13 or more.

WAIT, A QUESTION

Q *Isn't 7 million positions per second slow? I've heard of chess engines doing far more.*

A Positions per second depends on how much work each position needs. Here, every move updates 32 cells' patterns, threat bitboards and the network's accumulator. Depth is what counts, and the pruning tricks matter far more for depth than raw speed does.

6.4 State of the art

Every technique in this chapter, from LMR and null move to PVS, aspiration windows and Lazy SMP, is part of the standard toolkit of today's strong chess engines, and the chessprogramming community documents dozens of variations of each (Chessprogramming wiki, no date b; d). Many engines use

the same log-depth × log-move-number shape for LMR as this one (Chessprogramming wiki, no date b). What Gomoku engines add is threat awareness: Allis's threat-space search showed that forced sequences of fours and threes are where games are decided (Allis, Herik and Huntjens, 1996), and Gomoku engines still treat forcing lines specially for that reason. This engine's VCF quiescence and its threat-driven move narrowing are the Gomoku-specific part. The rest is borrowed from chess.

THE GOOD

+ Late moves cost a fraction of a full search; re-searches catch the surprises
+ Forced positions shrink from about 40 moves to a handful, or to one, by construction
+ VCF quiescence resolves four-sequences that a fixed horizon would cut
+ Lazy SMP scales with almost no synchronisation code

THE BAD

– Every reduction and pruning rule is a bet that's sometimes wrong
– Many tuned constants (margins, reductions, budgets), each needing match testing
– Lazy SMP wastes some work: threads often search the same nodes
– Search results become nondeterministic with more than one thread

KEY TAKEAWAYS

1. Ordering is everything: TT move, killers, history, then the learned policy.
2. LMR and pruning spend less depth on late moves and re-search when wrong.
3. Threat bitboards let the generator narrow forced positions to a few moves.
4. VCF quiescence follows sequences of fours past the horizon.
5. Lazy SMP: many threads searching the same thing, sharing a lockless table.

7 CHAPTER 7 The value network

IN THIS CHAPTER

7.1 The naive solution
- *hand-tuned numbers are guesses*
- *two microseconds is all we have*

7.2 The efficiently updatable trick
7.3 How it really works
7.4 State of the art

"NNUE (ƎUИИ Efficiently Updatable Neural Network) is, broadly speaking, a neural network architecture that takes advantage of having minimal changes in the network inputs between subsequent evaluations."
– nnue-pytorch documentation (nnue-pytorch contributors, 2026)

? **THE CRUX**

How do we judge a position the search didn't play out to the end, well enough to pick good moves, and cheaply enough to do it 7 million times per second?

THE SIMPLE VERSION

An experienced player can glance at a board and say "white is better" without calculating anything. They've seen thousands of games and recognise the shapes. The value network is that glance: it learned from hundreds of thousands of games which shapes tend to win.

HISTORY: THE ORIGINAL PROBLEM

For decades, evaluations were written by hand, like the classical one in chapter 4: a person picks the shapes and guesses their values. In 2018 Yu Nasu, writing for computer shogi, described the **NNUE**, a neural network designed so that a move only changes a few numbers inside it (Nasu, 2018). It was ported to Stockfish in 2019 and became Stockfish's default evaluation with version 12 in September 2020 (The Stockfish team, 2020; nnue-pytorch contributors, 2026).

7.1 The naive solution

We already have a naive solution: the hand-written evaluation from chapter 4. It adds up pattern values (1 for a B1 … 400 for an F5), threat bonuses (2,500 for our open four) and capture terms. It plays decent Gomoku.

What we're ignoring is that every number in it is a guess. Is an open three worth 160 or 220? Does a B3 next to a capture threat count double? Does having 3 captured pairs change what an open four is worth? The classical evaluation only knows the interactions someone thought to write down.

> **NEW CONSTRAINT: HAND-TUNED NUMBERS ARE GUESSES**
>
> Each weight in `eval.zig` was set by a person and adjusted by trial. There are about 50 of them. The real value of a position depends on how shapes combine, which no one can write down rule by rule.

The obvious upgrade is to learn the evaluation from data. Take a deep convolutional network, as AlphaZero did (Silver *et al.*, 2018), feed it the 19×19 board, and train it to predict who wins.

> **NEW CONSTRAINT: TWO MICROSECONDS IS ALL WE HAVE**
>
> At 7 million positions per second on 16 threads, each evaluation gets about 2 microseconds of CPU time, and most of that is already spent on the board update and the search itself. A deep convolutional network does many millions of multiply-adds per evaluation, which on a CPU takes far longer than that.

7.2 The efficiently updatable trick

Here's the NNUE idea. Make the first layer of the network a sum of rows from a big table, one row per **active feature**:

$$A = \sum_{\text{features } f \text{ present}} T[f]$$

where $T[f]$ is a row of 16 numbers learned for feature f. Computing A from scratch means adding hundreds of rows. But a move only changes a handful of features, so from one position to the next, A changes by "subtract the rows of features that disappeared, add the rows of features that appeared". The expensive layer is updated incrementally, exactly like the board in chapter 3. Everything after it is a tiny network, cheap enough to run in full.

The question is what the features should be. This engine's answer, in `nnue.zig` and `train_value.py`, builds on chapters 3 and 4. For **every empty cell** q of the board:

- four **window** features: the raw 16-bit window of q in each direction, its 8 neighbours along that line. That's one row of a 65,536-row table per direction;
- one **cell-state** feature: the cell's Pattern4 category for both colors, whether each color can capture there, and whether each would be a forbidden double three there. That's $13 \times 13 \times 2 \times 2 \times 2 \times 2 = 2{,}704$ possible states.

So the accumulator is a sum of five rows per empty cell. It only looks at empty cells, which makes sense for Gomoku: a position's value lives in where the players can play next, and the empty cells near stones are exactly where the patterns of chapter 4 live.

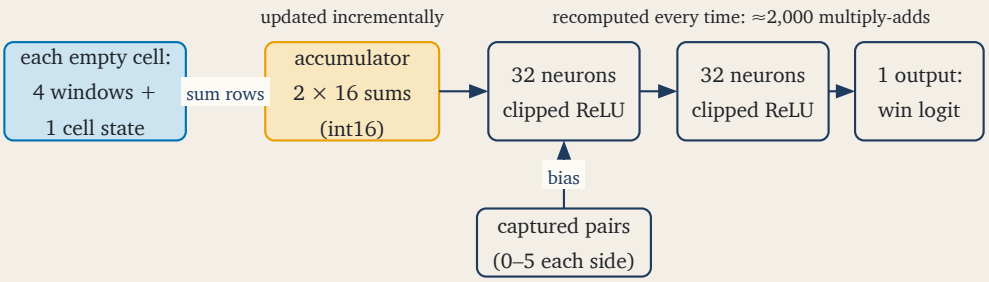


FIGURE 12 *The value network. The big part (left) is a sum of table rows, kept up to date as stones come and go; the small part (right) runs in full at each evaluation. The 16 sums exist twice, once for each player's point of view.*

HOW MUCH THE TRICK SAVES

How few rows actually change? When a stone lands on a cell, that cell stops being empty: its five rows leave the sum. Then each of the 32 neighbours on its lines (Figure 4) gets a new window in one direction, and maybe a new cell state. Each change subtracts the old row and adds the new one, so that's at most $5 + 2 \times 32 + 2 \times 32 = 133$ row operations, all folded into one delta vector and added once (`reclassifyNeighbours`). Recomputing from scratch would add five rows for every empty cell: about 1,800 rows early in the game.

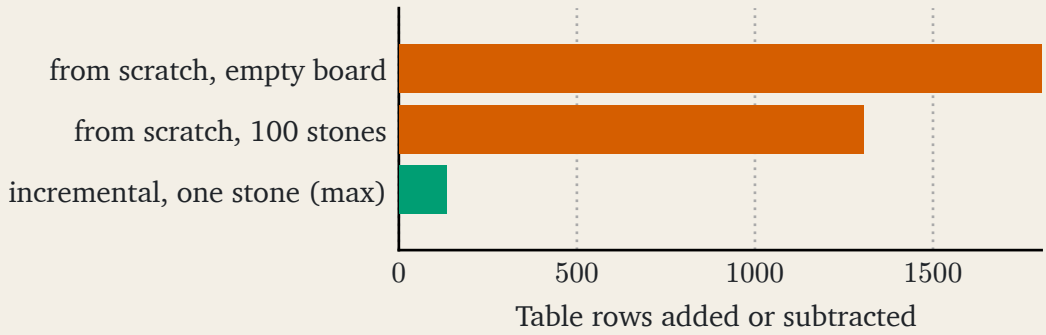


FIGURE 13 *Row operations per position, computed from the feature definition (a capture adds the same work again per captured stone). Incremental updates are 10–14 times cheaper than recomputing, before counting that they're folded into a single vector add.*

TWO POINTS OF VIEW

Whose turn it is matters enormously in Gomoku: an open four is a win for the side to move and a near-certain loss for the other. So the accumulator is kept twice: once seen from black (black's stones are "own"), once from white, with colors swapped in the windows and cell states. Each table row stores both halves side by side, so one row load updates both. At evaluation time, the side to move's half goes first. The network always sees "me" and "them", never "black" and "white".

WAIT, A QUESTION

Q *If a cell's patterns are stored for both colors, why does the network need two perspectives?*

A Because "black can make an open four here" and "white can make an open four here" mean opposite things depending on whose turn it is. The tables store facts; the evaluation decides what they're worth to the side to move.

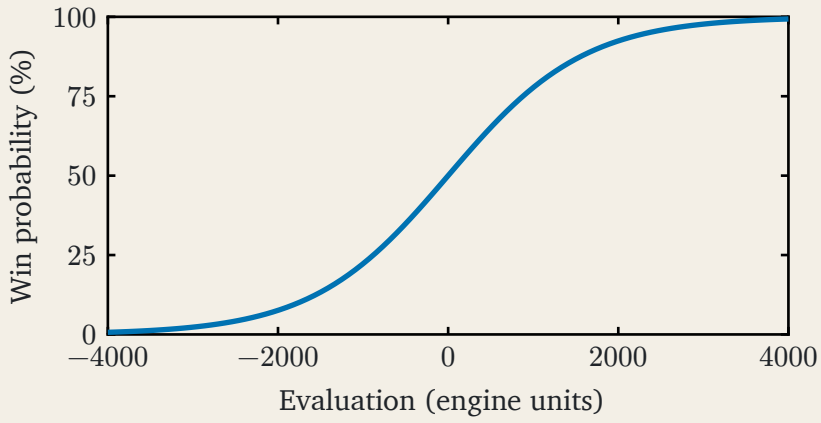


FIGURE 14 *How the engine reads its own scores: win probability* $= \sigma(\text{score}\ /800)$. *A score of +800 means about 73%; +2,400 about 95%. The side panel's win probability in the UI is this curve.*

THE HEAD AND THE OUTPUT

The 32 accumulator values (16 for each side) pass through a **clipped ReLU** into two hidden layers of 32 neurons each, then one output. Captured pairs don't fit naturally into the per-cell features, but they matter a lot (five pairs wins), so they enter as a learned bias on the first hidden layer: one row of 32 values per captured-pair count, for each side.

The output is a **logit** of "the side to move wins". The engine multiplies it by a scale $K = 800$ so it lands in the same units as the search's scores. Read backwards, a score s means a win probability of $\sigma(s/K)$, as in Figure 14.

WAIT, A QUESTION

Q *The window table has 65,536 rows of 32 numbers. Isn't that a big network?*

A It's big in memory (about 4 MB per network) but tiny in compute. Each evaluation only touches the rows that changed. The parts that run in full are the two hidden layers, about 2,000 multiply-adds. A convolutional net like AlphaZero's does many millions per evaluation.

SYMMETRY FOR FREE

A Gomoku position is just as good rotated or mirrored, and there are 8 such symmetries. The trainer **ties** each window's row to its mirror image, and the same table serves all four directions. Rotating or reflecting the board permutes the directions and mirrors some windows, which leaves the sum unchanged. So the network is exactly invariant to all 8 symmetries by construction, without any data augmentation.

7.3 How it really works

Let's put numbers on it, from the shipped `assets/standard.nnue`:

- accumulator width 16 per perspective (`C = 16` in `train.sh`), so each row is 32 int16 values;
- 65,536 window rows and 2,704 cell-state rows, about 4.37 MB, nearly all of the 4.4 MB file;
- head: 32 → 32 → 32 → 1, int8 weights, clipped ReLU between layers;

- 12 capture-bias rows (0 to 5 pairs, for each side);
- output scaled by 800 and clamped to ±19,000, below the mate threshold.

Notice where the 4.4 MB goes: almost all of it is the window table, and almost none of it is touched per evaluation.

One network is trained per rule variant: standard, without captures, without the double-three ban, and without both. The rules change what shapes mean (a B4 that can be captured isn't much of a four), so each variant gets a network trained on its own self-play games. All four are embedded in the binary; a variant without a network falls back to the classical evaluation.

WAIT, A QUESTION

Q *Wouldn't a bigger, smarter network play better?*

A Not necessarily, and the author tested it. A 22-million-parameter transformer trained on the same games predicted results much better (validation loss 0.282 against 0.334 for the int8 network), but it's far too slow to call at every node. Using it as a teacher to train the small network gave a slightly lower loss and no measurable Elo gain. For an alpha-beta engine, the evaluation has to be fast first and smart second.

7.4 State of the art

NNUE is now the standard evaluation in strong alpha-beta engines. Stockfish's classic NNUE used much larger feature sets, based on the king's square and every piece's square, and a 256-wide accumulator per side, with about 10 million parameters in the first layer, but the shape is the same: a big sparse, incrementally updated first layer, then a small dense head computed in integers (nnue-pytorch contributors, 2026). In Gomoku, Rapfi's network, Mixnet, also starts from line patterns: it looks up each point's four line segments in precomputed tables, updates the result incrementally, and runs in 16-bit and 8-bit integers. Its authors report that it needs orders of magnitude less computation than larger residual networks, with accuracy between that of a small 4-block and a 6-block one, and that it beats the GPU-based Katagomo when no GPU is available (Jin, Duan and Hang, 2025). At the other end, AlphaZero and KataGo use deep residual convolutional networks that see the whole board at once (Silver *et al.*, 2018; Wu, 2019). They're much more accurate per evaluation, much slower, and paired with MCTS, which needs far fewer evaluations.

THE GOOD

+ Learned from hundreds of thousands of games instead of guessed
+ ≤ 133 row operations per stone instead of about 1,800 for a full recompute
+ Exactly symmetric under the 8 board symmetries by construction
+ Fast enough for every node of a 7 M/s search

THE BAD

– Only sees each line within 4 cells; no global view of the board
– Small capacity: a 22-million-parameter transformer predicts results much better
– One network per rule variant to train, check and ship
– The board and trainer must compute the features identically, forever

KEY TAKEAWAYS

1. NNUE's first layer is a sum of table rows, so a move updates it by adding and subtracting a few rows.
2. Features here are, for every empty cell, its four line windows and its rules-aware state.
3. The network always sees "me versus them", so one set of weights serves both colors.
4. The network is symmetric by construction and outputs a win logit scaled by 800.

8 CHAPTER 8 Integers all the way down

IN THIS CHAPTER

8.1 The naive solution
- *rounding errors add up*
- *16-bit sums overflow*
- *the CPU wants 8-bit times 8-bit*

8.2 How it really works

8.3 State of the art

"The convolution involves 8-bit integer operands and a 32-bit integer accumulator."
– Jacob *et al.* (2018)

THE CRUX

How do we run a network trained with floating-point numbers using only small integers, and get exactly the answer the trainer computed?

THE SIMPLE VERSION

Imagine drawing a plan on graph paper where you're only allowed to put points on the grid lines. If you design on the grid from the start, nothing is lost when you hand it over. If you design freehand and then snap every point to the grid, the drawing gets a little wrong everywhere. The trick in this chapter is to train the network on the grid from the start.

HISTORY: THE ORIGINAL PROBLEM

Neural networks train with 32-bit floats, but phones and CPUs run 8-bit integer arithmetic many times faster. In 2018, Jacob and colleagues at Google described how to train networks that run with integer arithmetic only, by simulating the rounding during training (Jacob *et al.*, 2018). The rounding has no useful gradient, so training uses the **straight-through estimator**, which pretends rounding is the identity when computing gradients (Bengio, Léonard and Courville, 2013).

8.1 The naive solution

Train the network in floating point. When it's done, multiply every weight by a scale, round to the nearest integer, and write the integers to a file. The engine then computes with integers and divides by the scale at the end. This is post-training quantization.

We're ignoring that rounding errors add up, that integers overflow, and that the CPU's fast instructions want very particular sizes.

NEW CONSTRAINT: ROUNDING ERRORS ADD UP

The accumulator sums about 1,800 rows. Each rounded value is off by up to half a unit, so the sum can drift by tens of units, and the drift differs from position to position. The engine's integer network would quietly disagree with the network that was trained and measured.

Two fixes work together. First, reparametrise so the most common rows cost nothing. The all-empty window and the plain empty cell show up hundreds of times per position. The trainer forces their rows to zero and moves the empty cell's value into a bias multiplied by the number of empty cells, computed once in higher precision. Second, use **quantization-aware training** (QAT): for the last 4 of its 24 epochs, the trainer's forward pass rounds every table value, weight and activation exactly as the engine will (Jacob *et al.*, 2018). The network learns weights that work well after rounding.

NEW CONSTRAINT: 16-BIT SUMS OVERFLOW

The accumulator lives in int16, half the size of int32, so twice as many values fit in each SIMD register and each cache line. A sum of 1,800 rows can easily pass 32,767.

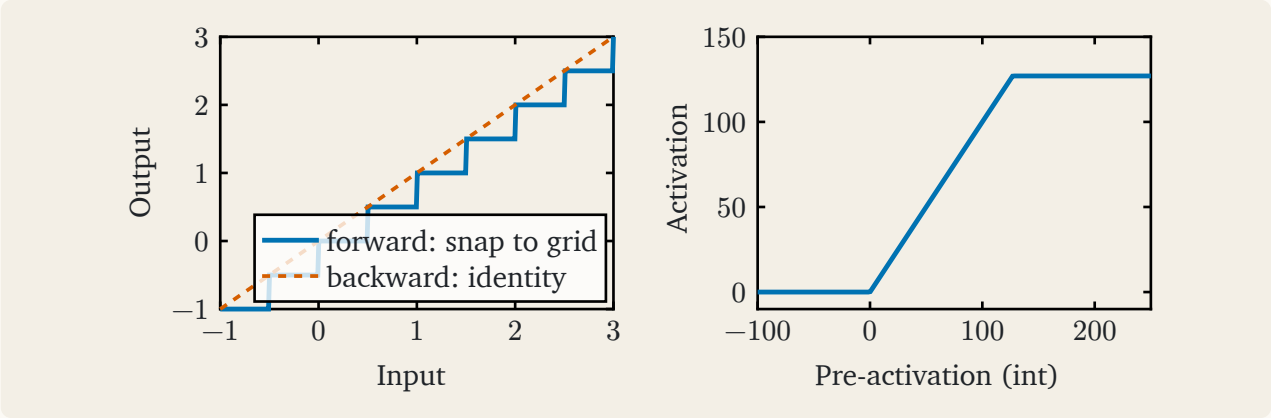


FIGURE 15 *Left: the straight-through estimator. The forward pass uses the staircase, exactly what the engine computes; the backward pass pretends it was the dashed line so gradients still flow. Right: the clipped ReLU, which clamps every activation to 0…127 so it fits in a signed 8-bit integer.*

The fix is pleasingly mathematical. The board keeps the accumulator **relative to the empty board** (the trainer computes the empty board's sums and the engine subtracts them), and it adds with wrapping arithmetic. Wrapping addition is arithmetic modulo 2^{16}, so intermediate overflows don't matter: as long as the final value fits in int16, it comes out right, no matter how the sum wandered on the way. The exporter picks the table scale Q, a power of two, as large as possible while keeping the largest difference seen in the data within ±30,000 (`qlog_for` in `train_value.py`).

NEW CONSTRAINT: THE CPU WANTS 8-BIT TIMES 8-BIT

Modern CPUs have dot-product instructions that multiply 8-bit integers in bulk and add into 32 bits (`sdot` on ARM). They're only usable if both the activations and the weights are 8-bit.

So the head is built for them. Activations are clipped to 0…127 (the clipped ReLU in Figure 15), weights are int8 at a scale of 64, so each weight lies in $\pm 127/64 \approx \pm 1.98$, and each layer is an int8 × int8 → int32 dot product followed by a shift right by 6. Scales are powers of two so that dividing by them is a shift, not a division. The final output is multiplied by 800 and shifted right by 13.

WAIT, A QUESTION

Q *Why not just use floats in the engine? CPUs are fast at floats too.*

A Two reasons. Speed: 8-bit dot products process four times as many values per instruction as 32-bit floats. And exactness: with integers and power-of-two scales there's no rounding ambiguity at all, so the engine can prove it computes the same function as the trainer, which floats can't promise across compilers and CPUs.

8.2 How it really works

The pipeline from trainer to engine has an explicit check, and it's the part to remember:



FIGURE 16 *From float weights to a shipped network. The last arrow is the guarantee: a single mismatch on the 64 check positions means the engine's features or arithmetic differ from the trainer's, and the network isn't shipped.*

1. `train_value.py` trains in fp32 for 20 epochs, then turns on QAT for the last 4. In QAT, table values live on the grid $1/Q$, biases on $1/(64Q)$, activations are floored to multiples of $1/128$ and weights rounded to multiples of $1/64$, all with straight-through gradients.
2. `export.py` writes the integer tables and weights to the `GNN3` file, computes the output of its own integer reference implementation on 64 validation positions, and writes those positions and scores next to the network (`.check.bin`, `.check`).
3. `eval.sh` builds the engine with the new network embedded and runs `Gomoku --nncheck` on those 64 positions. The engine builds each position on its own incremental board and evaluates it. The result must be "0/64 mismatches"; otherwise the author doesn't ship it.

So the guarantee is: the integer network in the engine computes **exactly** the function that was measured on validation data during training. One subtle detail the code comments mention: the large per-empty-cell term goes through 64-bit floats in the trainer, so the trainer's accumulator lands on the same grid as the engine's integers.

8.3 State of the art

This is the same recipe Stockfish uses: an int16 accumulator, clipped ReLU activations in 0…127, int8 weights in the dense layers, power-of-two scales, and a trainer, `nnue-pytorch`, that clamps weights to the ranges quantization allows (nnue-pytorch contributors, 2026). The design follows from the constraint, not from taste: the dot-product instructions every modern CPU has are 8-bit, and an evaluation called millions of times per second can't afford anything else. The bit-exact export check is less common. Many projects accept small float-versus-integer differences. Here, any mismatch also catches bugs in the board's incremental features, which makes it a very good test.

THE GOOD

+ Four times more values per SIMD instruction than fp32
+ Exact: the engine computes the very function measured during training
+ Wrapping int16 accumulator: no overflow handling in the hot loop
+ The exactness check doubles as a test of the board's feature code

THE BAD

− Weights are limited to ±1.98 and activations to 0…127: less expressive
− QAT and reparametrisation make the trainer considerably more complex
− Any change to features must be mirrored, bit for bit, in Python and Zig

KEY TAKEAWAYS

1. Rounding after training drifts; training with simulated rounding (QAT) doesn't.
2. The straight-through estimator lets gradients pass through rounding.
3. Wrapping int16 arithmetic relative to the empty board makes overflow harmless.
4. int8 × int8 → int32 dot products with power-of-two scales map onto CPU dot-product instructions.
5. 64 check positions must match exactly, or nothing ships.

9 CHAPTER 9 The learned policy

IN THIS CHAPTER

9.1 The naive solution
- *hand-made ordering is another pile of guesses*
- *ordering has to cost no more than before*

9.2 How it really works

9.3 State of the art

"Instead of a handcrafted evaluation function and move ordering heuristics, AlphaZero uses a deep neural network (p, v) = fθ(s) with parameters θ."
– Silver *et al.* (2018)

THE CRUX

How do we guess which moves deserve to be searched first, using what the engine learned from its own games instead of hand-made rules?

THE SIMPLE VERSION

Before you think hard about a position, a little voice says "try that one". It's usually right, and it saves you from thinking about silly moves first. The policy is that little voice, trained by watching which moves the full search ends up choosing.

HISTORY: THE ORIGINAL PROBLEM

Move ordering decides how much alpha-beta prunes (Knuth and Moore, 1975), so engines have always invested in it: killer moves, the history heuristic (Schaeffer, 1989), and hand-

written rules ("fours first, then threes"). AlphaZero went the other way: a **policy** output of its network predicts the probability of each move, and the tree search spends its effort according to it (Silver *et al.*, 2018). This engine borrows the idea, in a form cheap enough for alpha-beta.

9.1 The naive solution

The classical engine orders moves by hand (`handcraftedScore` in `movepick.zig`): each Pattern4 category gets a value for what the move makes for us and another for what it takes away from the opponent. Making a four scores high, blocking an open three scores high, and so on. We're ignoring two things: that these values are guesses, and that whatever replaces them must cost no more per candidate.

NEW CONSTRAINT: HAND-MADE ORDERING IS ANOTHER PILE OF GUESSES

Is blocking the opponent's three better than making our own? Does a cell that threatens a capture deserve a bonus? It depends on the position, and a fixed table of category values can't say.

The fix is to **learn** the ordering. The training data from chapter 10 records, for every position, the move the search eventually chose. So we can train a model to predict it: give every candidate cell a score, turn the scores into probabilities with a softmax, and minimise the cross-entropy against the search's choice.

NEW CONSTRAINT: ORDERING HAS TO COST NO MORE THAN BEFORE

The ordering score is computed for every candidate at every node, dozens of times per node. A neural network per candidate would cost more than everything else combined.

So the policy is a **linear** model over things the board already stores for each cell. For a candidate cell q, from the side to move's point of view:

$$s(q) = P_c[\text{cell state}(q)] + \sum_{d=1}^{4} L[\text{line info}(q,d)] + C_{\text{own}}[\ldots] + C_{\text{opp}}[\ldots]$$

- P_c: one entry per cell state, the same 2,704 states the network uses;
- L: one entry per direction for the joint line information of both colors, the 16-bit `LineInfo` pair from chapter 4;
- C_{own}, C_{opp}: the captures available at q for each side, crossed with how many pairs each side already has (capturing your fifth pair wins; capturing your first doesn't).

That's seven table lookups and six additions per candidate, on data already sitting in the cell's 16-byte record. It's no more expensive than the hand-made ordering it replaces.

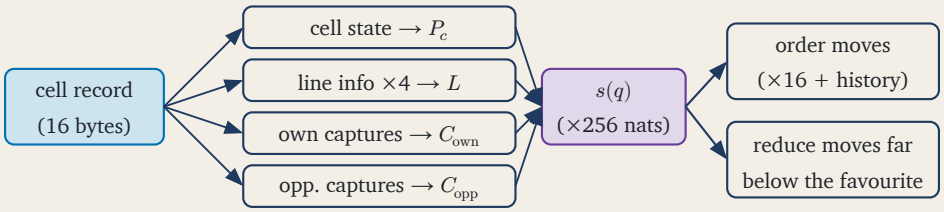


FIGURE 17 *The learned policy is a sum of table lookups on the cell record the board already keeps. Its score is used twice by the search: to order moves, and to decide how much to reduce the ones it dislikes.*

WAIT, A QUESTION

Q *A linear model? Isn't that too simple to be useful?*

A It's simple in the sense that matters for speed, but its inputs are rich: the line-info lookups already encode patterns and captures for both colors in each direction, and the 2,704 cell states encode their combination. It gets the search's top move right about 44% of the time. A 22-million-parameter transformer trained by the author reached about 76%, but costs millions of operations per position. The engine needs the cheap one.

9.2 How it really works

`train_policy.py` trains the tables on the same self-play records as the value network. For each position, the candidates are exactly the engine's candidate cells (empty, within the 5×5 square or three steps along a line from a stone, and not a forbidden double three when the ban applies). The loss is a softmax cross-entropy against the move the search chose. Four epochs are enough. The tables are exported as int32 values scaled by 256, so a score is a log-probability in units of 1/256 of a **nat**. It's a linear model, so the integer version matches the float one up to rounding.

The search uses the score in two places (`policy.zig`, `Tuning`):

1. **Ordering.** The policy score, multiplied by 16, replaces the hand-made score and is added to the history bonus. TT moves and killers still go first.
2. **Reductions.** For each move, the search computes how far its policy score drops below the best move at this node, and adjusts the LMR reduction: one ply **less** for the policy's favourite, unchanged up to 4 nats below it, one ply more from 4 nats, two plies more from 6 nats. Four nats is a factor of about 55 in probability.

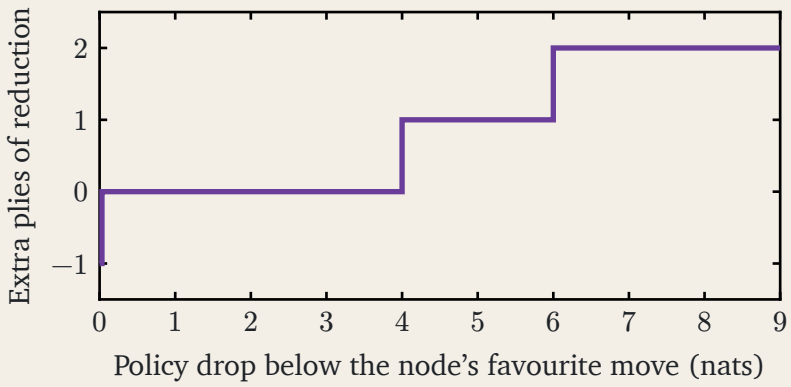


FIGURE 18 *How the policy adjusts late move reductions, computed from* `clamp((drop − 512) / 512, −1, 2)` *in units of 1/256 nat. Only the favourite itself (drop 0) gets the bonus; moves the policy finds at least* $e^4 \approx 55$ *times less likely than the favourite get searched shallower.*

The author also tried using the policy to **prune** moves outright at low depth. It lost Elo, so it's off (`prune_margin = 0`). With ordering and reductions, the policy measured +29 Elo at equal time, with a 95% confidence interval of [+1, +57]. That's a modest gain, but it comes nearly for free.

WAIT, A QUESTION

Q *Why is the policy trained on the search's choice rather than on the move that won the game?*

A Because the search's choice is a much cleaner signal. A game has one winner but dozens of moves, many of them irrelevant to the result. The move a deep search picked in a position is a direct statement about that position, and it's what the ordering is trying to predict anyway: "which move will the search end up liking?"

9.3 State of the art

In MCTS engines like AlphaZero and KataGo, the policy is a head of the same large network as the value, and it's essential: it decides which branches the search explores at all (Silver *et al.*, 2018; Wu, 2019). In alpha-beta engines, ordering traditionally comes from search statistics (TT, killers, history) rather than from a learned model. Rapfi's small network has a policy head next to its value head, computed from the same incrementally updated pattern features (Jin, Duan and Hang, 2025). This engine sits in between: a learned policy, but linear and built from lookups, so it costs no more than the hand-made ordering it replaced, and the search's own statistics still go first.

THE GOOD

+ Learned from the search's own choices, not guessed
+ Seven lookups per candidate: same cost as hand-made ordering
+ Also drives reductions: unlikely moves are searched shallower
+ Integer tables match the trained float model up to rounding

THE BAD

– Linear: picks the search's top move only about 44% of the time
– Using it to prune hurt, so its role is limited to ordering and reductions
– The gain is modest: +29 Elo, with a wide confidence interval

KEY TAKEAWAYS

1. The policy predicts which move the search will prefer, so the search can try it first.
2. It's a linear model: seven table lookups on the cell record, scaled log-probabilities.
3. It's trained with a softmax over the engine's own candidate cells.
4. The search uses it for ordering and to reduce moves far below the favourite: +29 Elo.

10 CHAPTER 10 Self-play and Elo

IN THIS CHAPTER

10.1 The naive solution
- *a game result is a noisy label*
- *self-play games all look alike*
- *lower loss is not more wins*

10.2 How it really works

10.3 State of the art

"AlphaZero learns these move probabilities and value estimates entirely from self-play; these are then used to guide its search in future games."
– Silver *et al.* (2018)

THE CRUX

Where do the training positions come from, and how do we know that a new network actually makes the engine stronger?

THE SIMPLE VERSION

Practise chess against yourself, and after every game write down every position along with who won and what you thought of each position at the time. After thousands of games, you have a notebook that says which positions tend to win. Then, to see whether you got better, play a match against your old self.

HISTORY: THE ORIGINAL PROBLEM

Learning from games played against oneself is old. AlphaZero made it famous in 2018 by learning chess, shogi and Go from self-play alone, starting from random play (Silver *et al.*, 2018). Engines like Stockfish train their NNUE networks by supervised learning on engine-evaluated positions mixed with game results, a cheaper loop than reinforcement learning from scratch (nnue-pytorch contributors, 2026). To compare versions, everyone measures playing strength in **Elo** points, a scale Arpad Elo designed for chess players (Elo, 1978).

10.1 The naive solution

Let the engine play itself. Record every position and, when the game ends, label every position with the result: $+1$ if the side to move went on to win, -1 if it lost. Train the network to predict that label. Then check that the validation loss went down.

We're ignoring how noisy those labels are, how alike self-play games are, and that a lower loss isn't the same as more wins.

NEW CONSTRAINT: A GAME RESULT IS A NOISY LABEL

A position 40 moves before the end gets the same label as the final one. If the winner blundered into a lost position at move 20 and the loser blundered back at move 35, every position in between has the wrong label.

The fix: mix the result with the search's own opinion. Every record also stores the score the search gave the position (searched to a fixed budget of 30,000 nodes). Turned into a probability with the same sigmoid as Figure 14, it's a much more local label. The training target is

$$y = \lambda \cdot \frac{\text{result} + 1}{2} + (1 - \lambda) \cdot \sigma(\text{score}\ /K)$$

with $\lambda = 0.5$ and $K = 800$. This is the same idea as Texel's tuning method for classical evaluations, which fits $\sigma(\text{score}/K)$ to game results (Chessprogramming wiki, no date e).

NEW CONSTRAINT: SELF-PLAY GAMES ALL LOOK ALIKE

A deterministic engine playing itself from the empty board plays nearly the same game every time. Thousands of copies of one game teach the network nothing.

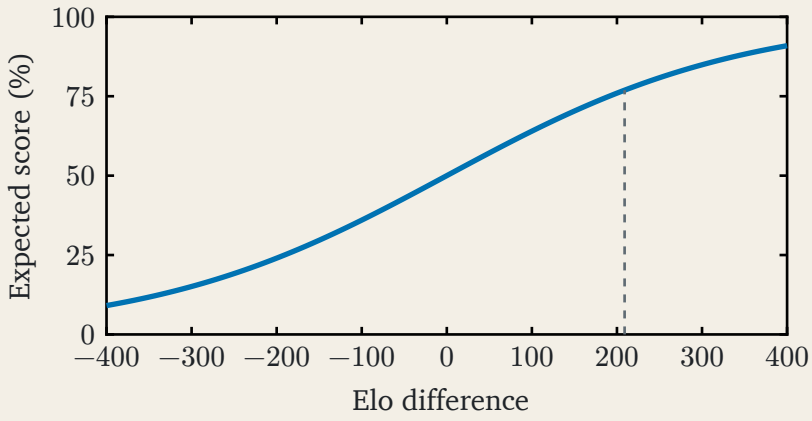


FIGURE 19 *What an Elo difference means. The dashed line is the shipped network's +209 Elo over the classical engine at equal time: an expected score of about 77%, roughly three points out of every four games.*

The data generator (`src/tools/datagen.zig`) injects variety in two places. Each game starts with a random opening of 1 to 8 stones: the first near the centre, the rest among candidate cells. Then, during the first 16 plies, the engine plays a random move among those within 40 points of the best 35% of the time. That keeps the games diverse while keeping the moves reasonable.

NEW CONSTRAINT: LOWER LOSS IS NOT MORE WINS

The validation loss measures how well the network predicts labels. The engine's strength depends on how its errors interact with the search, with speed, and with move ordering. A network can predict better and play worse.

So the final test is always a match. `eval.sh` plays the new engine against the classical one (hand-written evaluation, no policy), in game pairs: each opening is played twice with colors swapped, so neither side benefits from a lucky opening. It plays once at equal time (100 ms per move) and once at equal nodes (20,000 per move), and reports the Elo difference with a 95% confidence interval.

WAIT, A QUESTION

Q *Why play in pairs with colors swapped?*

A Gomoku strongly favours the first player, and random openings can be lopsided. If an opening is a near-forced win for black, playing it once would hand a free point to whoever got black. Playing it twice with colors swapped cancels that out.

10.2 How it really works

So how does one generation actually get made? Here's the whole loop, as the scripts in `train/` run it:

1. **Generate** (`gen_data.sh`): the current engine, with its current networks embedded, plays hundreds of thousands of games, one single-threaded engine per core, 30,000 nodes per move. Each position becomes a 104-byte record: the board at 2 bits per cell, side to move, captured pairs, result, search score, best move.

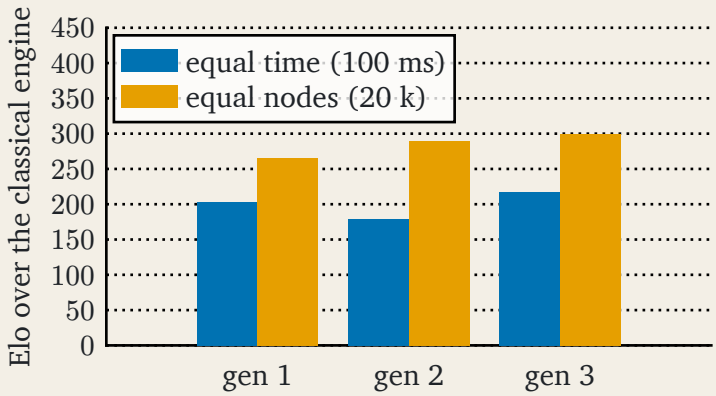


FIGURE 20 *Elo of three generations of the standard-rules network against the classical engine, from the author's match logs. At equal nodes the gain keeps growing with each generation; at equal time it's flatter, because the network costs time per node. (The gen-2 time match was disturbed by other load on the machine.)*

2. **Train** (`train.sh`): the value network for 24 epochs (QAT in the last 4), with the AdamW optimiser (Loshchilov and Hutter, 2019) and a one-cycle learning-rate schedule (Smith and Topin, 2019), then the policy for 4. The last two data files of each directory are held out for validation.
3. **Export** (`export.py`): integer files plus the 64 check positions.
4. **Evaluate** (`eval.sh`): the exactness check, the policy's accuracy, and the two matches.
5. **Ship**: only if it's better and the benchmark still answers every position in under 500 ms, copy the files into `assets/`.

The order matters: nothing is shipped on loss alone, only on matches. Then repeat: the new engine generates the next generation's data. Generation 0 was played by the classical engine, about 12.4 million positions. Each later generation, about 13 million positions, was played by the networks trained on the previous ones.

Figure 20 shows the lesson of this whole document in one chart. **At equal nodes**, the network's judgement keeps improving: +266, +290, +300. At equal time, the gains are smaller and flatter, because evaluating the network costs time that the classical engine spends searching deeper. Strength is a product of judgement and speed, and the design of every chapter here is about keeping both.

TRAINING FAST ON A GPU

One detail of `train_value.py` is worth knowing. A position's accumulator is a sum of rows, so it can be written as a count vector (how many times each window and cell state appears) multiplied by the table. The trainer precomputes, for every position, its list of distinct windows and cell states with their counts, stores those on the GPU, and computes the first layer of a whole batch as one dense matrix product. That's how it trains on about 6 million positions per second on a single GPU, and why 13 million positions per generation is plenty.

WAIT, A QUESTION

Q *Isn't this just copying the search? How can the network end up better than the engine that made its data?*

A It's a bootstrap. The labels come from a search, and a search with a weak evaluation still sees deeper than the evaluation alone. The network learns to predict, in one glance, what took that search 30,000 nodes. Then the next generation's search is built on that better glance, and its labels are better again. The gains shrink each round, but they compound.

10.3 State of the art

There are two families of training loops. AlphaZero and KataGo use reinforcement learning from scratch: MCTS self-play with a large network, where the search's visit counts train the policy and the game results train the value (Silver *et al.*, 2018; Wu, 2019). It's very strong and very expensive: AlphaZero generated its self-play games on 5,000 TPUs (Silver *et al.*, 2018). Stockfish-style NNUE training is cheaper: generate positions with an existing engine, label them with its evaluations and the game results, and train the small network by supervised learning (nnue-pytorch contributors, 2026). This engine follows the second family, starting from the classical engine as generation 0. Rapfi adds a third ingredient: it first trains a larger convolutional network, then distils it into its small pattern-table network (Jin, Duan and Hang, 2025).

THE GOOD

+ Each generation needs only a CPU run for data and a short GPU run for training
+ Mixed labels (result and search score) are much less noisy than results alone
+ Match testing at equal time and equal nodes separates judgement from speed
+ Starts from an existing engine: no learning from random play

THE BAD

− The network inherits the blind spots of the search that labelled its data
− Gains shrink each generation: +266 → +290 → +300 at equal nodes
− Elo intervals are wide unless you play hundreds of game pairs
− Four rule variants means four data sets, four trainings, four evaluations

KEY TAKEAWAYS

1. The engine plays itself with random openings and some early randomness; each position becomes a 104-byte record.
2. The target mixes the game result with the search's score through $\sigma(\text{score}/800)$.
3. Matches against the classical engine, in color-swapped pairs, are the only real test.
4. Equal-node Elo measures judgement; equal-time Elo measures what you actually get.

11 CHAPTER 11 State of the art review

IN THIS CHAPTER

11.1 Why alpha-beta with a small network
11.2 What each piece is worth
11.3 Where the frontier is

Suppose all you had was a laptop and half a second per move. Which of the known ways to build a strong engine would you pick, and why? Now that every piece is in place, let's put this engine next to the systems it borrows from. There are three serious ways to build a strong board-game program today, and they differ mainly in where they spend their computation: on searching more positions, or on judging each one better.

	This engine	Rapfi	Stockfish	AlphaZero / KataGo
Game	Gomoku with captures	Gomoku, Renju	Chess	Chess, shogi, Go
Search	alpha-beta, PVS, VCF quiescence	alpha-beta	alpha-beta, PVS	MCTS
Evaluation	NNUE on line windows + cell states	Mixnet: line-segment tables + small head	NNUE on king and piece squares	deep residual CNN
Move ordering	TT, killers, history, linear policy	policy head + search heuristics	TT, killers, history	policy head of the CNN
Arithmetic	int8 / int16, CPU	int8 / int16, CPU	int8 / int16, CPU	floating point, GPU / TPU
Training	supervised on self-play with search labels	distilled from a larger CNN	supervised on search-labelled positions	reinforcement learning from scratch

TABLE 2 *Four designs side by side (Silver* et al.*, 2018; Wu, 2019; Jin, Duan and Hang, 2025; nnue-pytorch contributors, 2026). The CPU engines share almost every choice; the real split is between small fast evaluations with a big alpha-beta search, and big slow evaluations with a selective MCTS.*

11.1 Why alpha-beta with a small network

Every design choice in this engine follows from three constraints: a CPU, a 500 ms budget, and a very tactical game.

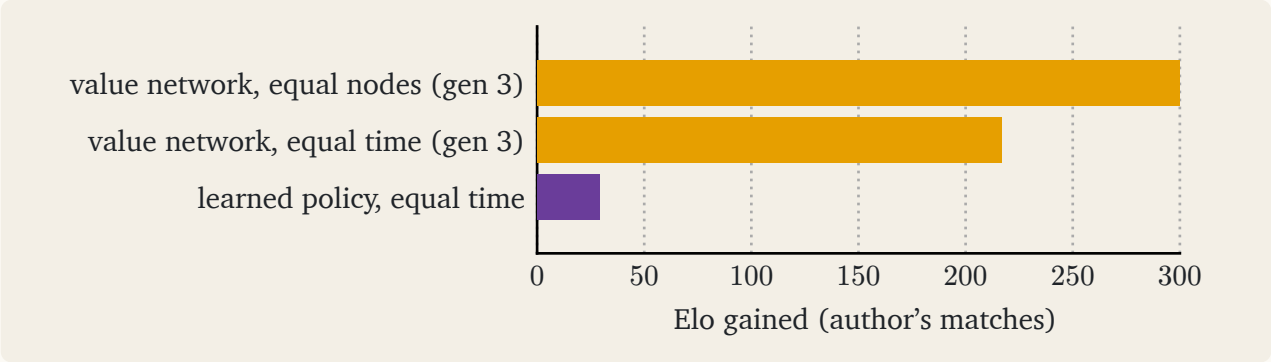


FIGURE 21 *Measured contributions of the model, in Elo over the classical engine. Equal-node numbers flatter the network because they hide its cost per node; equal time is what you get when playing. (The README's figure for the shipped network is +209 at equal time over 800 games.)*

- **A tactical game punishes missed replies.** In Gomoku a single unanswered four loses on the spot. Alpha-beta with threat-driven move generation and VCF quiescence proves those lines exactly, which is why both Gomoku engines in Table 2 use it, and why Allis's solution of the game was built on threat search (Allis, Herik and Huntjens, 1996).
- **A CPU and 500 ms leave about 2 μs per position.** That rules out deep networks at every node, and NNUE was designed for exactly this regime (Nasu, 2018). MCTS with a big network needs far fewer evaluations, but each one needs a GPU to be fast (Silver *et al.*, 2018).
- **Strength is judgement times speed.** Figure 20 showed it: the network's judgement is worth 300 Elo at equal nodes, but only about 200 of those survive at equal time, because evaluating costs time.

11.2 What each piece is worth

Figure 21 puts the model's measured contributions side by side. The value network is the big win, and the policy is a small but nearly free extra.

11.3 Where the frontier is

The obvious next step would be a smarter evaluation, and the author's experiments mark where that stops paying. A transformer with 22 million parameters predicts game outcomes much better than the int8 network (validation loss 0.282 against 0.334) and picks the search's move 76% of the time against 44% for the linear policy. Distilling it into the small network gave a slightly lower loss but no measurable Elo. The small network's capacity, not the quality of its labels, is what limits it.

So the frontier for an engine like this is the trade between capacity and speed: a slightly wider accumulator or hidden layer that pays for its time, better features, or using a big network's knowledge in ways that don't cost time per node. Rapfi's work suggests the trade can be pushed further with careful architecture (Jin, Duan and Hang, 2025). MCTS engines in the KataGo family sit at the other end of the same trade, and with a GPU they can be stronger still (Wu, 2019).

THE GOOD

- + Runs anywhere with a CPU; one binary, networks embedded
- + Tactically exact where Gomoku needs it: forced lines are proved
- + Every learned component is checked bit-exact and match-tested

THE BAD

- – The small network's capacity caps how well it can judge
- – Many hand-tuned search constants remain
- – Without a GPU, the MCTS + deep network path is closed

Glossary

bitboard
A set of board cells stored as bits of machine words, so that set questions ("is any cell set?") become a few word operations.

clipped ReLU
An activation that clamps values to a fixed range, here 0 to 127, so they fit in an 8-bit integer.

cutoff
Abandoning the remaining moves of a node because one move already proved the opponent will avoid this position.

Elo
A rating scale where a difference of D points predicts an expected score of $1/\left(1+10^{-D/400}\right)$ for the stronger player.

five-point
An empty cell where one more stone completes five in a row.

iterative deepening
Searching depth 1, then 2, then 3… until time runs out, keeping the result of the last completed depth.

killer move
A quiet move that caused a cutoff at the same depth in a sibling branch; likely to cut off here too.

late move reductions
LMR: searching moves that come late in the ordering to a reduced depth, and re-searching at full depth only if they surprise us.

Lazy SMP
A parallel search where every thread runs the same iterative-deepening search on the same position, communicating through the shared transposition table.

logit
The input of a sigmoid: a logit z corresponds to the probability $1/(1+e^{-z})$.

mailbox
A board array with extra border cells around the real board, so that walks along lines hit a "wall" value instead of needing bounds checks.

Minimax
Choosing moves by assuming both players play their best: maximise over our moves, minimise over the opponent's, recursively.

nat
The natural unit of information: a difference of 1 nat in log-probability means a factor of $e \approx 2.72$ in probability.

negamax
A way of writing minimax with one function: a score for one side is the negation of the score for the other, so "min" becomes "max of negated children".

NNUE

Efficiently Updatable Neural Network: an evaluation network whose first layer is a sum of table rows, so a move updates it by adding and subtracting a few rows.

Pattern4

The engine's combined verdict for an empty cell and a color, over all four line directions: from "five" and "open four" down to "two" and "nothing".

ply

One move by one player. "Depth 10" means 10 plies: five moves each.

principal variation search

PVS: search the first move with the full alpha-beta window and the others with a zero-width window that only answers "better or not?"; re-search if the answer is "better".

quiescence search

A small search run at the leaves of the main search that only considers forcing moves, so positions are evaluated only once they're "quiet".

straight-through estimator

A training trick: apply rounding in the forward pass but pretend it was the identity in the backward pass, so gradients still flow.

threat-space search

Allis's method of searching only sequences of threats (fours and threes) that the defender must answer, which shrinks the game tree enormously.

transposition table

A large hash table, keyed by the position's hash, that remembers search results (score, depth, best move) so that positions reached again are not searched again.

VCF

Victory by Continuous Fours: a winning sequence where every attacking move is a four, so each defender reply is forced.

Zobrist hash

A position fingerprint made by XOR-ing one random 64-bit number per (stone, cell) pair; placing or removing a stone is a single XOR.

References

Allis, L.V. (1994) *Searching for Solutions in Games and Artificial Intelligence*. PhD thesis. Rijksuniversiteit Limburg. Available at: https://project.dke.maastrichtuniversity.nl/games/files/phd/SearchingForSolutions.pdf (Accessed: November 7, 2026).

Allis, L.V., Herik, H.J. van den and Huntjens, M.P.H. (1996) "Go-Moku Solved by New Search Techniques," *Computational Intelligence*, 12(1), pp. 7–23. Available at: https://doi.org/10.1111/j.1467-8640.1996.tb00250.x.

Bengio, Y., Léonard, N. and Courville, A. (2013) *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. Available at: https://doi.org/10.48550/arXiv.1308.3432.

Chessprogramming wiki (no date b) *Late Move Reductions*. Available at: https://www.chessprogramming.org/Late_Move_Reductions (Accessed: November 7, 2026).

Chessprogramming wiki (no date d) *Lazy SMP*. Available at: https://www.chessprogramming.org/Lazy_SMP (Accessed: November 7, 2026).

Chessprogramming wiki (no date c) *Null Move Pruning*. Available at: https://www.chessprogramming.org/Null_Move_Pruning (Accessed: November 7, 2026).

Chessprogramming wiki (no date a) *Principal Variation Search*. Available at: https://www.chessprogramming.org/Principal_Variation_Search (Accessed: November 7, 2026).

Chessprogramming wiki (no date e) *Texel's Tuning Method*. Available at: https://www.chessprogramming.org/Texel%27s_Tuning_Method (Accessed: November 7, 2026).

Donninger, C. (1993) "Null Move and Deep Search: Selective-Search Heuristics for Obtuse Chess Programs," *ICCA Journal*, 16(3), pp. 137–143. Available at: https://doi.org/10.3233/ICG-1993-16304.

Elo, A.E. (1978) *The Rating of Chessplayers, Past and Present*. London: Batsford. Available at: https://catalog.hathitrust.org/Record/000215863 (Accessed: November 7, 2026).

Gomocup (2026) *Gomocup 2026, the 27th tournament (June 5th–7th, 2026)*. Available at: https://gomocup.org/results/gomocup-result-2026 (Accessed: November 7, 2026).

Hyatt, R.M. and Mann, T. (2002) "A Lockless Transposition-Table Implementation for Parallel Search," *ICGA Journal*, 25(1), pp. 36–39. Available at: https://doi.org/10.3233/ICG-2002-25104.

Jacob, B. *et al.* (2018) "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 2704–2713. Available at: https://doi.org/10.1109/CVPR.2018.00286.

Jin, Z., Duan, H. and Hang, Z. (2025) *Rapfi: Distilling Efficient Neural Network for the Game of Gomoku*. Available at: https://doi.org/10.48550/arXiv.2503.13178.

Knuth, D.E. and Moore, R.W. (1975) "An Analysis of Alpha-Beta Pruning," *Artificial Intelligence*, 6(4), pp. 293–326. Available at: https://doi.org/10.1016/0004-3702(75)90019-3.

Loshchilov, I. and Hutter, F. (2019) "Decoupled Weight Decay Regularization," in *7th International Conference on Learning Representations, ICLR 2019*. New Orleans, LA, USA: OpenReview.net. Available at: https://openreview.net/forum?id=Bkg6RiCqY7 (Accessed: November 7, 2026).

Nasu, Y. (2018) *NNUE: Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi*. Appeal document, 28th World Computer Shogi Championship. Available at: https://www.apply.computer-shogi.org/wcsc28/appeal/the_end_of_genesis_T.N.K.evolution_turbo_type_D/nnue.pdf (Accessed: November 7, 2026).

nnue-pytorch contributors (2026) *NNUE*. Available at: https://github.com/official-stockfish/nnue-pytorch/blob/master/docs/nnue.md (Accessed: November 7, 2026).

Reinefeld, A. (1983) "An Improvement to the Scout Tree Search Algorithm," *ICCA Journal*, 6(4), pp. 4–14. Available at: https://doi.org/10.3233/ICG-1983-6402.

Renju International Federation (1998) *International Rules of Renju*. Available at: https://www.renju.net/rifrules/ (Accessed: November 7, 2026).

Schaeffer, J. (1989) "The History Heuristic and Alpha-Beta Search Enhancements in Practice," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(11), pp. 1203–1212. Available at: https://doi.org/10.1109/34.42858.

Shannon, C.E. (1950) "Programming a Computer for Playing Chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314), pp. 256–275. Available at: https://doi.org/10.1080/14786445008521796.

Silver, D. *et al.* (2018) "A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play," *Science*, 362(6419), pp. 1140–1144. Available at: https://doi.org/10.1126/science.aar6404.

Slate, D.J. and Atkin, L.R. (1977) "CHESS 4.5 – The Northwestern University Chess Program," in P.W. Frey (ed.) *Chess Skill in Man and Machine*. Berlin, Heidelberg: Springer, pp. 82–118. Available at: https://doi.org/10.1007/978-3-662-06239-5_4.

Smith, L.N. and Topin, N. (2019) "Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates," in T. Pham (ed.) *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*. SPIE (Proceedings of SPIE), p. 1100612. Available at: https://doi.org/10.1117/12.2520589.

The Stockfish team (2020) *Stockfish 12*. Available at: https://stockfishchess.org/blog/2020/stockfish-12/ (Accessed: November 7, 2026).

Wikipedia contributors (2026) *Pente*. Available at: https://en.wikipedia.org/w/index.php?title=Pente&oldid=1383374870 (Accessed: November 7, 2026).

Wu, D.J. (2019) *Accelerating Self-Play Learning in Go*. Available at: https://doi.org/10.48550/arXiv.1902.10565.

Zobrist, A.L. (1970) *A New Hashing Method with Application for Game Playing*. Technical report 88. Madison, Wisconsin: Computer Sciences Department, University of Wisconsin. Available at: https://minds.wisconsin.edu/handle/1793/57624 (Accessed: November 7, 2026).